Ricardo Melo Czekster



Markov Chains for programmers

2nd Edition: October/2026

Markov Chains for programmers

Second Edition – October/2026

Ricardo M. Czekster
Birmingham, United Kingdom



Self Publishers Worldwide

This book was typeset using LaTeX software (using OverLeaf – online).
It employs proprietary software (mostly MS-Excel) for demonstrating MC.
It uses the PRISM Probabilistic Model Checker, version 4.6 (2022).
For demonstrating direct solution methods, it works with MATLAB®.
LLMs were used to fix typos and inconsistencies, only lightly.

Copyright © 2026 Ricardo M. Czekster
License: Creative Commons Attribution 4.0 International (CC BY 4.0)

Preface

This book originated as I perceived a need for better and simpler explanations surrounding Markov Chains (MC). If you take the formal literature on this subject and if you do not have a rather strong mathematical background to understand the concepts, you are probably restricted to use MC, which I always thought that I could change that. In my view, this is a nice opportunity to sharp programming skills as the numerical methods proposed pose interesting challenges.

The purpose of this book is to present MC firstly to programmers (at any level), but it is not restricted though; I think that broader audiences might enjoy it as well. The idea is to grasp the basic notions and then implement solutions that are reproducible. Proof of that is that I have shared C code in GitHub and supporting spreadsheets so there are multiple ways of verifying answers.

I hope you enjoy the next pages. Please, feel free to send me your remarks, suggestions, comments, and (eventual) critics.

I shall keep track of changes among editions, errata, etc., in an adequate rate.

- First edition of the book was published in early April/2022 and it covered MC (types DTMC and CTMC), basic notions, and introductory models.
- Second edition was published in October/2026 and it fixes typos and errors I (and my preferred LLM engine) found throughout the years.

Preamble to 2nd edition

I surrendered to LLMs in early 2026 and their ability to humble humans. When I prompted my favourite engine that time, it yielded 30 mistakes, minor typos, inconsistencies, and code/model mistakes after a 1-min "analysis". This edition is thus a new version of all these modifications for my audience.

All LLM-generated texts and insertions are marked in the book as grayed out, just like this sentence here.

2nd October, 2026
Birmingham, United Kingdom
Ricardo M. Czekster

Table of Contents

Introduction **1**

1 Markov Chains **3**
1.1 Model and system . . . 3
1.2 Emergence of Markov Chains . . . 3
1.3 Basic modelling primitives . . . 4
1.4 Input and output . . . 4
1.5 Types . . . 5
1.6 Values decorating transitions . . . 5
1.7 Markov property or memoryless property . . . 7
1.8 Let's code! . . . 9
1.8.1 Challenge 01 . . . 9
1.8.2 Challenge 02 . . . 10
1.9 What is considered a 'proper' MC? . . . 11
1.9.1 Irreducible MC . . . 12
1.10 Limitations of Markov Chains . . . 13
1.11 Comments on the spreadsheets . . . 13

2 DTMC **15**
2.1 Remembering matrix operations . . . 16
2.2 The Belfast weather model . . . 17
2.3 Let's code! . . . 17
2.3.1 Challenge 03 . . . 18
2.4 Solution methods for DTMC . . . 20
2.4.1 Power matrix, or Matrix-Matrix Multiplication . . . 20
2.4.2 Vector-Matrix Multiplication . . . 20
2.4.3 Forward simulation . . . 21
2.4.4 Useful comments . . . 22
2.4.5 Direct solution method . . . 22
2.4.6 PRISM DTMC model . . . 23
2.5 Let's code! . . . 24
2.5.1 Challenge 04 . . . 24
2.5.2 Challenge 05 . . . 26

2.6 Comments on the spreadsheets . . . 26

3 CTMC **29**
3.1 Infinitesimal Generator . . . 29
3.2 Computing the uniformised DTMC . . . 31
3.2.1 Computing the embedded DTMC . . . 32
3.3 The Lily Pad model . . . 33
3.4 Let's code! . . . 34
3.4.1 Challenge 06 . . . 34
3.5 Solution methods for CTMC . . . 35
3.5.1 Forward simulation . . . 35
3.6 Race condition . . . 36
3.7 Comments on the spreadsheets . . . 38
3.8 Let's code! . . . 38
3.8.1 Challenge 07 . . . 38
3.9 Solution methods (cont.) . . . 40
3.9.1 Direct solution method . . . 40
3.9.2 PRISM CTMC models . . . 41

4 More projects and models **43**
4.1 Projects . . . 43
4.1.1 Combine it all! . . . 43
4.1.2 Visual MC . . . 44
4.1.3 Parallelisation of simulation samples . . . 44
4.2 Models . . . 44
4.2.1 Birth and Death model . . . 44
4.2.2 Mouse Maze model . . . 45
4.2.3 Availability model . . . 46
4.2.4 Aging and Rejuvenation model . . . 47

5 Final considerations **49**
5.1 What's next? . . . 49

About the author **51**

Index **55**

List of Figures

1.1 A simple CTMC model with two states and two transitions. 6
1.2 An example of periodic MC with three states. 11
1.3 Two MC for demonstrating irreducibility (in this case, the lack thereof) properties (note that Q^0 and Q^1 are not irreducible). 12

2.1 A simple DTMC with 2 states (the "lighting model"). 15
2.2 Model representing the weather of Belfast, in Northern Ireland. . . . 18

3.1 Representation of balancing rates in states in CTMC. 29
3.2 Lighting model with two states as a CTMC. 30
3.3 Generic CTMC model with four states. 30
3.4 Lily Pad model showing possible pads and transitioning rates. 33
3.5 Hypothetical situation showing the problem of choosing the next state in CTMC simulation. 37
3.6 Drawing pseudo-numbers from uniform and exponential distributions to help simulate time in CTMC ($\log$ is the natural logarithm, in base e). 38

4.1 A four state Birth and Death CTMC model. 44
4.2 A mouse in a maze visiting cells mindlessly. 45
4.3 Availability modelling of a blade server in a data centre. 46
4.4 Software aging and rejuvenation model [1]. 47

List of Tables

1.1 Duration and rate for decorating transitions in CTMC models. 6
1.2 Yearly observations of a person waiting for a bus. 8

2.1 A two state representation of the lighting problem. 15

3.1 Observed time until each transition from each lily pad, and the corresponding rates. 34

Introduction

Markov Chains is a recurrent topic in systems analysis. The vast literature on the subject and applicability attests its importance and timeliness. A host of authors have delved into the formalism, proposing new modelling primitives, or enhancing numerical tractability throughout the years. However, if one wants to learn about its fundamental underpinnings, however simple, the overwhelming number of outcomes prevents others from deepening the knowledge. Another problem is that sometimes the wealth of previous results about Markov Chains focuses on specific audiences, e.g., statisticians, mathematicians, and so on, seldom on computer programmers.

We aim to bridge this gap in this work, by presenting Markov Chains in simple terms and then tackling the programming issues within the formalism. This book is dedicated to programmers and people initiating studies in Markov Chains. It aims to expose Markov Chains to programmers (at any level) having elementary mathematical background and willingness to delve into this modelling and analysis approach.

If you are looking for more formal aspects, please look elsewhere in the vast literature on this subject. This book is about understanding the basic principles surrounding Markov Chains and how to implement basic numerical methods to derive actionable indices for analysis. It has been written for programmers or engineers with enthusiasm for computing (and coding), eager to get their hands dirty on Markov Chains and willing to grasp results in short time.

In pursuing this objective, I shall:

- Employ as little mathematical notation as possible.
- Propose modelling with a corresponding spreadsheet or code (usually in C in this edition).
- Simplify and convey notions with matching implementations.

The book does not address the mathematical foundations of Markov Chains and intricate solution mechanisms such as Chapman-Kolmogorov equations, Perron-Frobenius theory, Krylov sub-spaces, Chebyshev methods, Arnoldi or GMRES solution methods, and other similar concepts. Please, do submerse in this literature (elsewhere) if it is relevant to you. There are a lot of interesting notions lurking in previous research and, depending on your background, it might prove useful in your career. For example, you could start looking at this book by Häggström [2] or Trivedi & Bob-

bio work on Reliability and Availability engineering [3] which are both remarkably interesting.

Before we embark any further in this journey, a few housekeeping, and notes of caution:

- From time to time, there will be programming 'challenges'; the idea is that one tries to code and test and come up with one's own solution. Solutions to all challenges are in the GitHub repository (folders `project00` and `project01`); most of the code is written in the C Programming Language.
  - All code is covered by the GPLv3 License – free to redistribute citing the source. Feel free to use it as you wish.
  - For the spreadsheets MS-Excel was used – do test it on other platforms.
  - GNU/Linux is the recommended OS for testing the software provided here – it was tested on a Virtual Machine running KUbuntu 18.04 and GNU/Linux kernel 5.4.100(generic).
- There is a website that accommodates all models, auxiliary code, and spreadsheets available at: `https://github.com/czekster/markov` – clone it and compile the projects thereof (the book is CC-BY-4.0 and the code is GPLv3).
- At all times, try to fully understand the code, *what* it does, *how* it does, and *why* it does. Understanding the sequences of commands is crucial to improve reasoning about the subject.

Enjoy.

Chapter 1

Markov Chains

This is a gentle introduction to Markov Chains (MC). Before delving into MC however, we need to comment on basic key notions and ideas.

1.1 Model and system

A system, broadly speaking, comprises a set of interacting components that collaboratively perform functions to address a problem. A model abstracts a system, i.e., it captures the most relevant operational semantics (i.e., the essence of what it effectively does) into constructs for example a table (in mathematical words, in a matrix) of relationships among pre-modelled components. Here, it is true that the adage *"modelling is more of an art than a science"* comes up. Good modellers learn how to capture the system's function in essence and convey it into a working model that not only represents its behaviour, but it is amenable for reasoning and decision making.

Norbert Wiener (1894-1964), a mathematician, once stated that *"The best material model for a cat is another, or preferably the same cat"*. If your model represents the totality of parts belonging to a system under study, then why modelling? Why not inspecting and observing the real-world system for extracting properties and inferences?

1.2 Emergence of Markov Chains

Markov Chains (MC) were envisioned by Andrey Andreyevich Markov (1856-1922). His doctoral advisor was P. Chebyshev[1]. He published the first paper on the topic in 1906 and then applied later to study the distribution of vowels in Alexander Pushkin's opus *Eugene Onegin*. During those days, it attracted a reasonable amount of attention. Markov himself thought of the limited applicability of his result, perhaps only to the analysis of texts and identification of author's styles.

[1] Pafnuty Lvovich Chebyshev was a founder of the St Petersburg school of mathematics.

It was picked up by Alan Scherr in the 1960's in his thesis on time shared systems (MIT), where he applied to study scalability issues [4]. Nowadays, analysts employ MC to study a variety of phenomena, from economic models to Internet searching (PageRank® algorithm [5] [6]). The following paper [7] details the top five most significant MC applications, with interesting discussions (in information theory, in indexing the Internet, and so on).

1.3 Basic modelling primitives

MC models employ states, transitions, and values decorating transitions (I will comment more about these 'values' in Section 1.5). And that's it. It cannot be any simpler than that. The problem in MC is not about the primitives but about the abstractions modellers use to convey behavioural ideas to their investigations. You must make decisions on what and about the Level-of-Detail (LoD) of components, in a way that is not too abstract nothing is learned, and not too detailed that hinders comprehension.

There is a wealth of literature on MC [8, 9, 10], numerical methods [11], and applications [12], relating models to systems, decomposition techniques, division of responsibilities, coupling/cohesion, and so on. The vastness of notions should not overwhelm you because after all, what is important here is to capture the operational essence of systems and key component interactions.

1.4 Input and output

MC will take a model of a system with enumerable sets of states and transitions decorated with values as a Directed Graph (DG), i.e., a construct that has vertices or nodes (states), connected by arcs (transitions). A DG connects states where every transition has arrows pointing to other states (also known as a *digraph*).

After one finish working on the model, tools prepare the model for numerical solution by applying procedures that creates matrices with interesting properties (to be inspected and discussed later). The procedure also checks whether the MC respects fundamental properties (mostly ergodicity – explained later as well) or checking impossible MC models (models having invalid states or values – these are *absorbing* states, which need different solution methods and are not covered in this book (see Section 1.9)).

Even if a model is well-formed, it may not yield a solution in a timely fashion (steady state analysis vs transient analysis that will be discussed later). What analysts would like to compute are the permanence probability of each state in the model, i.e., the percentage of time spent in that state in a very long simulation. The computation is done by different ways, for instance, one might employ simulation, direct solution (using a set of linear equations), numerical methods (power matrix, Vector-Matrix Multiplication), and so on. One computes the *eigenvector* of the matrix, i.e., the unique vector that 'describes' the matrix (let's put this way to simplify everything – I urge you to deepen your knowledge a bit more on this concept, in specialised literature). Another way of thinking about eigenvectors is to think that one

is performing a dimension reduction, from a matrix (a two-dimensional structure) to a one-dimensional vector. Again, we compute this vector only if the matrix respects a set of properties which we shall discuss in due time.

1.5 Types

The literature provides a distinction for Markov Processes and Markov Chains. It explains that modellers use the first for describing continuous processes whereas the second one for discrete time considerations. For the present work we will call Continuous Time Markov Chains (CTMC) when working with durations and rates when modelling states and Discrete Time Markov Chains (DTMC) when analysts would decorate transitions with probabilities [3].

Important

- **CTMC:** when working with durations (that are converted to rates), residence time in states (or sojourn times).
- **DTMC:** when decorating transitions with probabilities.

That is what will happen to the model, intuitively. A bit more formally, we care about observations of the system at time T. If these are discrete, e.g., $T = \{0, 1, 2, \ldots\}$, then we have a discrete stochastic (random) process whereas if T is continuous, where $T = [0, \infty)$ the process is continuous [11]. In CTMC, to proper work with the Markov property, the time spent in a state must be independent of the time already spent in that state. Thus, this time must be exponentially distributed whereas in DTMC, the residence time in a state that respects the Markov property must be ruled by a geometrically distributed value, which are the only distributions that exhibit the memoryless property [11].

1.6 Values decorating transitions

After discussing the types of MC, a lot of concepts *floated* around. One must decorate transitions with rates, frequencies, or durations for CTMC, and probabilities for DTMC. Let's start with the easiest way of decorating transitions, employing durations. Modellers should observe a system for a considerable amount of time and then inspect the amount of time the observed entity (it could be anything, it will be dependent on the model) has remained in *that* state. We say that the modeller is observing the *residence time in* the state (also called the *sojourn* time). Then, to decorate the transition, it will require to compute the *rate out* of the state (leaving the state) by using the inverse of the duration (i.e., $\texttt{Rate} = \frac{1}{\texttt{Duration}}$), also called the *frequency out* of the state [3]. Observe Table 1.1 next:

For DTMC, it is simpler: one decorates transition with perceived probabilities out towards other states or within the same state (loops). Loops are disregarded in CTMC for reasons that will be explained later (note for *impatient* readers: the diagonal is

Table 1.1: Duration and rate for decorating transitions in CTMC models.

Observed duration (time spent IN the state) in min	Rate or frequency (OUT of the state) over one hour	Remarks
60	$60/60 = 1$	If any given entity has stayed in the state for 60 minutes, the rate exiting this state in one hour will be one, i.e., it would leave the state once in one hour.
30	$60/30 = 2$	If in a state for 30 min, the rate out would be modelled as two.
Note that one could have used $\frac{1}{60}$ and $\frac{1}{30}$ to decorate transitions without any difference for the solution. That is because of, in terms of *proportions*, they are the same.		

obliterated and the negative line sum of the rest of transitions is used). Now, let's reason about a simple model with two states as depicted in Figure 1.1 (this shall be referred as "The lighting model"):

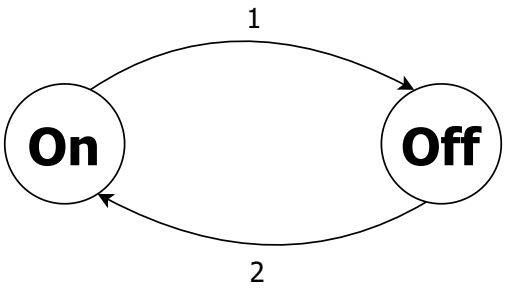


Figure 1.1: A simple CTMC model with two states and two transitions.

In this example, the modeller had observed that when a light switch was `On`, it remained lit for 60 minutes whereas when `Off`, someone would invariably turn back on every 30 minutes. As we shall discuss in detail next one could take this representation of the behaviour of a system and convert it to a table that will be subjected to a numerical method that will be able to answer the following: *"At any given time of the day, what is the probability of the system in the `On` state or in the `Off` state"*.

In other words: *"You were somehow 'teleported' into this room: what is the probability that the light is either `On` or `Off`?"*. Having the ability to answer this one might take measures to decide on making changes to the system or promoting ways to help users turn the light off more frequently to save money, and so on. Of course, this is a *very simple* model, and it respects all requirements of MC in terms of well-formed models, and well-behaved durations, and so on. We shall discuss cases where these assumptions may not hold true.

1.7 Markov property or memoryless property

By now you can identify different types of MC (continuous time or discrete time), however, deep down, what does this *actually* mean? MC have a governing principle that states that the past is irrelevant to determine the future steps taken by the system. Let's imagine a frog in a pond with a set of lily pads[2]. This mindless frog cannot store its past movements in its reduced brain capacity, so it jumps around lily pads never considering the visits taken beforehand. When jumping around, the frog has forgot about the past (perhaps because of its reduced brain capacity?), so it makes decisions on whether to stay or to move based on what it is perceiving in that moment in time. And then it keeps jumping between pads until it gets tired and go to sleep.

One observer (you?) might have been filming these for several hours and would like to know the set of lily pads that it most likely visits every time, by trying to extrapolate the frog's life until the infinite (looking for the steady state as one might say). So, you either take durations and mark the lily pads it visited (CTMC) or some probability given the observations (DTMC). And then you convert this system to a model and subject the model to a numerical method that will rank (by the highest probability), the set of most visited lily pads in the pond.

Now let's go back to those MC types, e.g., Continuous Time or Discrete Time once again, which is why we have started the discussion. Modelling with durations (and then with rates) and given the fact that you would like to simulate any given situation plus the fact that you are bounded by the memoryless property, you will have to model these durations also using a memoryless distribution. In this case, you would use the exponential distribution (mind that there are other distributions that are out of the scope of this work).

In this sense, you are modelling a continuous time 'evolution' (we might say) of this system. This distribution is better explained with an example. Let's suppose a person takes a bus to go home every day and after one year taking the bus and listing the time spent waiting for it in a bus stop, he (or she) ended up defining it as an exponential distribution of average parameter of 20 minutes. This means that one could wait one minute, 10 minutes, 25 minutes, or even 78 minutes (or more!) for the bus to arrive, just because it is using an exponential distribution. On *average* (if one draws successive samples) it will correspond to 20 minutes. See Table 1.2 (`spreadsheets/Chapter01-bus-times.xlsx`), with a list of some observations about the waiting time in the bus stop over a period of one year (365 observations).

To compute the time waiting in the bus stop for each day one uses the conversion from the uniform distribution (in MS-Excel it corresponds to the `RAND()` function, yielding uniformly distributed pseudo-random numbers between 0 and 1) to the exponential distribution: `Exp = -(PARAMETER) * LN(1-RAND())`. The `PARAMETER` value for this case was set to 20 (if one computes the average of draws, it will be closer to this value).

Note that this is because we are modelling backwards here: we are generating numbers when we already know the distribution (we do not have the actual real-world observations here – we are trying to demonstrate one possible way of deriving

[2] Attribution needed: the lily pads and frog example can also be found in [11], on page 4, detailing MC.

Table 1.2: Yearly observations of a person waiting for a bus.

	January	February	March	April	May	June	July	August	September	October	November	December
1	16.4	3.2	22.9	4.4	3.3	43.7	7.3	23.1	1.3	5.4	6.7	1.5
2	53.7	14.0	4.7	3.6	6.0	50.3	12.2	28.8	1.0	4.4	12.3	39.6
3	48.4	11.7	15.2	0.1	46.5	26.7	28.9	21.9	6.2	22.7	13.4	11.8
4	6.0	5.4	19.5	4.9	28.3	17.6	7.4	14.5	4.7	1.7	11.2	24.6
5	1.6	35.9	4.1	7.6	13.2	17.5	27.5	18.2	46.1	4.9	10.4	28.1
6	20.9	22.7	14.9	119.9	5.2	102.1	3.4	83.2	16.6	45.8	2.1	11.4
7	22.8	27.1	5.9	40.3	28.8	8.4	18.8	34.4	5.6	32.3	66.3	1.0
8	28.6	1.4	2.0	4.4	13.8	22.3	2.0	34.2	1.6	37.2	8.5	2.9
9	40.7	2.8	5.0	36.3	1.2	9.8	61.3	31.8	4.5	10.6	20.4	40.3
10	20.9	27.4	11.2	6.8	50.0	20.8	1.2	9.9	10.1	3.1	27.6	12.7
11	0.3	4.7	15.1	3.3	20.4	26.7	29.8	42.1	8.5	4.5	14.9	36.3
12	29.5	2.1	41.3	3.5	3.5	0.7	24.8	26.2	2.2	9.3	8.7	0.2
13	46.0	1.5	41.2	51.5	38.2	7.5	6.1	12.0	79.2	2.3	42.2	0.7
14	3.4	52.5	148.5	10.6	5.5	21.3	10.2	11.0	16.7	49.7	2.0	11.4
15	9.1	8.7	39.0	5.2	1.4	20.2	2.6	22.0	42.1	57.5	28.7	17.3
16	82.0	10.3	1.7	2.9	8.6	38.9	91.0	10.2	23.1	17.2	5.9	0.3
17	14.7	21.8	24.9	94.5	10.1	19.8	6.6	32.0	32.2	2.3	20.9	21.9
18	20.9	51.7	37.7	29.7	1.1	1.8	9.5	15.5	20.8	12.9	61.7	5.1
19	2.9	18.0	30.0	21.8	0.1	5.7	128.7	16.3	5.8	60.6	3.4	49.5
20	41.4	13.7	57.5	2.9	32.4	33.0	6.7	16.3	30.2	9.6	34.9	5.2
21	0.7	11.1	9.6	9.3	30.1	11.2	3.9	6.4	6.6	8.8	32.5	19.8
22	38.4	8.1	20.9	36.6	6.3	1.6	52.9	37.7	5.3	31.6	24.2	40.5
23	19.1	6.0	10.6	3.0	48.1	54.8	28.9	40.1	0.5	5.7	38.1	16.2
24	45.3	50.1	13.8	10.1	10.2	12.9	31.7	2.3	5.1	40.0	32.9	55.5
25	6.7	6.7	5.9	6.7	89.7	33.6	6.8	9.4	3.5	46.6	13.0	4.1
26	21.4	40.8	4.9	1.0	7.3	31.8	13.7	21.0	39.1	25.5	15.0	9.5
27	7.1	10.9	1.2	24.9	3.1	3.4	7.0	42.0	50.3	28.4	67.3	7.1
28	24.8	4.2	11.6	51.0	22.9	13.2	9.9	93.9	6.2	23.4	75.2	5.6
29	62.5		3.0	2.2	21.2	17.1	49.0	5.4	31.2	0.7	50.1	15.1
30	9.1		4.6	29.0	32.1	6.9	34.2	28.7	26.6	5.9	0.4	20.6
31	112.8		59.4		2.4		12.2	15.9		24.4		22.2
μ	27.7	17.0	22.2	20.9	19.1	22.7	23.8	26.0	17.8	20.5	25.0	17.3

these numbers). This is because one wants to work with the exponential distribution (at least for the time being).

What might happen in real life is that real measurements might not fit into the exponential distribution. In this case, we are in violation of employing MC to work with our problem and we shall resort to other technique, for instance, simulation. This shall be commented further in due time. For the time being, let's be happy with the fact that we can generate numbers from the exponential (memoryless) distribution. The last line (μ) in the table contains the monthly average.

If you take a closer look at this table, you will see that the average (for all table) is

21.7 minutes, the minimum value is 0.05 minutes, shown rounded as 0.1 in the table (3.2 seconds!) and the maximum value is 148.5 minutes (2.5 hours!!). So, there are good days and bad days, but on average, the expected behaviour is to approximate the mean I set forth: 20 minutes (i.e., a rate of $1/20$ per minute). It is true that it is memoryless, sometimes the bus took let's say 7 minutes (January 27th) but on January 28th, it took 24.8 minutes to arrive! As a matter of fact, it is capturing the memoryless property at its finest.

This is contrasted with Discrete Time. The best idea of understanding the concept is to consider a sort of *metronome*, with pulses ticking the same (without any losses). So, the system governed by a DTMC would change states following the rules set forth by the probabilities that decorated the transitions, without the notion of time passing (as it happened in CTMC).

To build a DTMC simulation, it suffices to model a situation where a visitor stands on a given state and draw a pseudo-random number from the uniform distribution and then use this value to consult and then jump to the next state. And then repeat this process many times and counts the visits to each state. Time is considered to jump on every metronome pulse, moment that the system changes state. We will stop now explaining about simulation, this topic is covered in detail in the next chapters.

1.8 Let's code!

For coding these challenges you will need to know how to process parameters from the command line and work with time constructs in C (for the pseudo-random[3] number generator).

1.8.1 Challenge 01

File: `challenge01.c`

Do: write a program that given a parameter N (passed in the command line), it computes and shows N uniformly distributed numbers (each line shows 10 numbers, to ease output).

Notes:

- The parameter (N) is an integer, but you will have to convert from the command line using the `atoi` function (this function converts a string to an integer).
- Use `#include <stdlib.h>` and `#include <time.h>` and auxiliary function `random()`.

[3] Most programming languages employ a pseudo-random generator that produces number from known formulas. See *Linear Congruential Generators* in the Rosetta code project: `https://rosettacode.org/wiki/Linear_congruential_generator#C`.

- Use a seed based on the current time (for increased randomness) with function `srand(time(NULL))`.

Good programming practice write a function to compute a uniform number and another function to show N uniform numbers, separating concerns in your code.

Suggested variations: (no code or auxiliary file will be provided)

- Do the same task in a spreadsheet (MS-Excel or other). Plot using a scatterplot. Plot the frequency on a bin size of 0.1. See that the numbers are *de facto* uniform.
- Count the number of values between 0.0 and 0.09, then from 0.1 to 0.19 and so on – they should have a uniform distribution, on average. See whether this is true.

1.8.2 Challenge 02

For the next challenge we shall use parts of the previous challenge (Section 1.8.1) to generate exponentially distributed numbers (adding the following 'cell' formula: `=(-1/A1)*LN(1-RAND())` – in MS-Excel, with parameter in cell `A1`)[4]. This challenge will use auxiliary mathematical libraries already available in most GNU/Linux distributions.

File: `challenge02.c`

Do: write a program that given a parameter N (passed in the command line) corresponding to the PARAMETER of the desired exponential distribution and another parameter M (the number of samples to compute), with functions to compute samples from the exponential distribution given the uniform distribution.

- The function prototype is `float exponential(float rate);`

Notes:

- The parameter (N) could be a float value, so conversions will use the `atof` function.
- You will use the `double log(double);` function in C.
  - So, you must compile the file with `-lm` (this will link your code with the math library).

[4]Note that now the *rate* is used, i.e., the formula takes $1/rate$ instead of the duration. The average of the samples approximates the mean duration $1/rate$, e.g., a rate of 0.05 per minute gives an average of 20 minutes.

Good programming practice: write *functions* as the previous challenge recommended.

Suggested variations: (no code or auxiliary file will be provided)

- Write the output to a file (text).
- Do the same task in a spreadsheet (MS-Excel or other). Plot using a scatterplot. Then order numbers and plot using lines, adding a trendline based on the exponential function.

1.9 What is considered a 'proper' MC?

Any matrix whose rows sum to 1 (DTMC) or to 0 (CTMC) defines a MC. However, for the solution methods in this book to produce a single, meaningful steady-state vector π, the chain must satisfy a few properties. We have discussed 'well-formed' or 'well-behaved' models in previous sections; now it is time to inspect what exactly this entails.

The first property is that the chain is *irreducible*: a visitor can get from every state to every other state, not necessarily in one step (see Section 1.9.1.). The second is that the chain is not *periodic*. A chain is periodic when returns to a state can only happen at multiples of some fixed number of steps $d > 1$. For example, consider the MC in Figure 1.2:

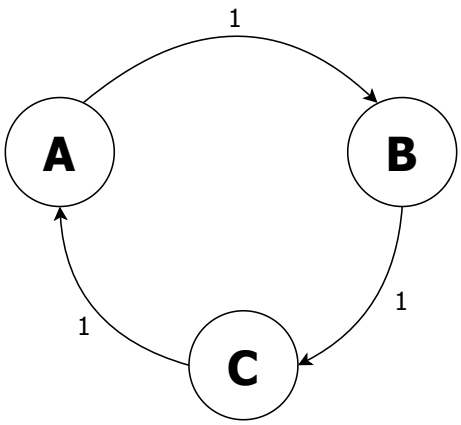


Figure 1.2: An example of periodic MC with three states.

Starting in state A one might return to state A after three hops (one to B, then to C, then back to A) every time. This characteristic defines a periodic MC, so we shall concern modelling chains that are aperiodic.

Finally, the chain must be *positive recurrent*: starting from any state, the chain returns to it with certainty, and the *expected* number of steps (or expected time) until it returns is finite. For the finite models in this book, this comes for free: every finite irreducible chain is positive recurrent, so in practice only irreducibility and aperiodicity need checking.

Important In summary, we are dealing with MC that are irreducible, aperiodic, and positive recurrent. For finite chains, irreducible and aperiodic together is what most books call *ergodic*, and it guarantees a unique π that the iterative methods converge to. CTMC cannot be periodic, so for them irreducibility is enough.

That's the mantra of MC in (almost every) book ever written about this subject. Start getting acquainted with the notion that not every MC you put down as a model will have a steady-state solution. If your MC adheres to these properties, discovering the solution vector shall be possible within a decent amount of time, using appropriate numerical methods. We will not cover here MC with absorbing states, i.e., states with no outgoing transitions to other states (in a DTMC, a self-loop with probability 1). It is worth noting that non-ergodic MC will not yield results if one employs a (let's say) classic numerical method. As we shall see these models might yield results if one resorts to simulation approaches.

1.9.1 Irreducible MC

Consider the next two MC, Q^0 and Q^1 depicted in Figure 1.3. In model Q^0, whenever state E gets visited, it stays switching between E and F states indefinitely. In terms of result vector, we shall see probabilities only for those two states.

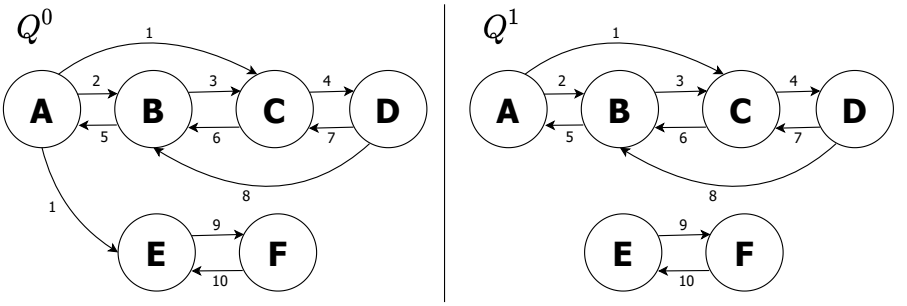


Figure 1.3: Two MC for demonstrating irreducibility (in this case, the lack thereof) properties (note that Q^0 and Q^1 are not irreducible).

In model Q^1, there are in fact two MC to consider, one with states A, B, C, and D, and another with E and F. Thus, one could solve each one separately, instead of it all. We are concerned here about chains that are irreducible, i.e., starting in any state, it

is possible to reach any other state (not necessarily in one step). If Q^1 had a transition from F to A, B, C, or D, it would be sufficient to deem the matrix irreducible, because it would respect the property.

William Stewart's book [11], on page 38, discusses so called *Nearly Completely Decomposable* (NCD) stochastic matrices, cases where transitions among states are weak (or low), where the matrix is irreducible however the model could have been broken in two (or more) to ease analysis. The literature refers to partitioning the state space in subsets with strong and weak interactions. Please, refer to the source for further information about NCD. Interested readers may also enjoy other properties such as *aggregation* or *lumpability* as well as other notions that surface as recurrent topics in MC.

1.10 Limitations of Markov Chains

MC are not free from drawbacks, unfortunately. For example, models should respect the memoryless property because *all* rates are drawn from the exponential distribution. Not all real-world problems are directly observed to respect this constraint, as other probability distributions could be present.

Another limitation concerns the state space of problems: even for simple models, the number of states required to represent the MC could be enormously high (state space explosion problem), presenting difficulties in achieving solution. This problem poses limits to MC as modellers should employ clever abstractions to represent large problems.

To cope with some of the limitations of pure MC and its dimensionality problems, research has proposed higher-level constructs such as *structured MC*. Examples are Queueing Networks (QN), Stochastic Petri Nets (SPN), Performance Evaluation Process Algebra (PEPA), or Reactive Modules (formalism employed by the PRISM Probabilistic Model Checker to represent CTMC, DTMC, Markov Decision Processes – MDP and Probabilistic Timed Automata – PTA) [13].

1.11 Comments on the spreadsheets

For this chapter we have the following spreadsheet to inspect:

- `spreadsheets/Chapter01-bus-times.xlsx`: it shows the exponentially distributed bus times using the uniform distribution.

Play with the spreadsheet, change parameters, see what happens, build code that replicates (note: it will never yield the same results[5] since you are employing different sets of pseudo-random numbers) the results of the spreadsheet.

[5] Unless you make the `seed` parameter to the `srand(int seed);` function constant.

Chapter 2

DTMC

Let's start discussing DTMC, where models have probabilities in the transitions, as depicted in Figure 2.1 (this model was covered and commented earlier, the only difference is that for the other one we used rates – CTMC):

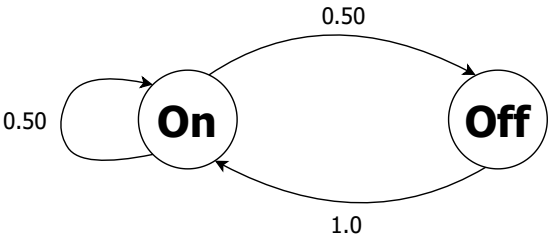


Figure 2.1: A simple DTMC with 2 states (the "lighting model").

For this MC, one may assume that since the On state is the most visited (because of the self-loop present in this model), the final probability vector π (our result) will be higher on this state (intuitively). The restriction of DTMC models is that they must have each line summing to 1.0, and for the case of numerical solution, that it also respects previously discussed MC properties.

There are several ways of solving this DTMC. For instance, after representing the model in a matrix, one could multiply it by itself many times, until it reaches convergence (steady state). This is known as the Power Matrix method. But first, let's create the model appropriately in Table 2.1:

Table 2.1: A two state representation of the lighting problem.

	On	**Off**
On	0.50	0.50
Off	1.00	0.00

After solution using the Power Matrix method (please, consult file in repository called `spreadsheets/Chapter02-DTMC-2states.xlsx`), we compute that $\pi_{\texttt{On}} = 0.66667$ (around 67%) and $\pi_{\texttt{Off}} = 0.3333$ (around 33%). Note that in the MS-Excel file we have employed the function `MMULT` for the same matrix. To do this, one must first select an empty 2x2 result cell (where the results will be), pressing F2, then putting the formula `=MMULT(D3:E4,D3:E4)` and pressing `CTRL-SHIFT-ENTER`. This is required in MS-Excel when operating with matrix multiplication (where the results are matrices as well). Each step squares the previous result ($M^2, M^4, M^8, \ldots$), so the power grows very quickly: after six *squarings* (M^{64}) every row equals π to more than ten decimal places. Multiplying by M one step at a time would need about 17 steps for six decimals. You may try to replicate these results as well.

Another way of reaching the same conclusions is to employ a Vector-Matrix Multiplication (VMM) method that from an initial vector multiplies the matrix iteratively until it finds the results. This method is more lightweight than the Power Method, because instead of a Matrix-Matrix multiplication, now it is a Vector-Matrix product, which is less cumbersome. It suffices to say that this method produces the same results as before. It took 21 steps for reaching the steady state.

Another difference in the MS-Excel for this method is needed: we should fix the matrix position (using the symbol '$' before cell placements). The initial vector chosen was `[ 1 0 ]` however, any initial vector summing 1.0 would work – perhaps the iterative method will have the side effect of performing better, i.e., producing the results in fewer iterations. Choosing a good starting vector (a "warm start") is one simple way of speeding up convergence; more powerful techniques, such as *preconditioning*, transform the problem itself, and there is a wealth of research on methods to accelerate the solution of MC. If the vector does not sum 1.0, you will have to adjust the results, because let's say, it sums 0.25, this 0.25 will be distributed two thirds to the `On` state (≈ 0.16667) and one third to the `Off` state (≈ 0.08333).

2.1 Remembering matrix operations

For MC solution mechanisms we will require to review matrix operations, especially matrix-matrix and vector-matrix multiplication. For example, when multiplying (operator 'x' below) two matrices X and Y, both of order 3x3, then $X \times Y$ is shown in Equation 2.1:

$$\texttt{Let } X = \begin{bmatrix} A & B & C \\ D & E & F \\ G & H & I \end{bmatrix} \text{ and } Y = \begin{bmatrix} J & K & L \\ M & N & O \\ P & Q & R \end{bmatrix}$$

Then,

$$X \times Y = \begin{bmatrix} AJ + BM + CP & AK + BN + CQ & AL + BO + CR \\ DJ + EM + FP & DK + EN + FQ & DL + EO + FR \\ GJ + HM + IP & GK + HN + IQ & GL + HO + IR \end{bmatrix} \quad (2.1)$$

There are several APIs for handling matrix multiplication efficiently. We are showing the basic process here because we are interested in coding this down the line (for our next model). As we have shown, MS-Excel (and other spreadsheet-based applications) have built-in functions to help multiplying matrices altogether. For Vector-Matrix Multiplication, the process is easier, i.e., only the first line of X is used (Equation 2.2):

$$\texttt{Let } X = \begin{bmatrix} A & B & C \end{bmatrix} \text{ and } Y = \begin{bmatrix} J & K & L \\ M & N & O \\ P & Q & R \end{bmatrix}$$

Then,

$$X \times Y = \begin{bmatrix} AJ + BM + CP & AK + BN + CQ & AL + BO + CR \end{bmatrix} \tag{2.2}$$

This concludes the basic reviewing of matrix operations that we will require to code the solution.

2.2 The Belfast weather model

This model is depicted on William Stewart's work [11] (page 6), where it models the weather of Belfast, Northern Ireland[1]. Figure 2.2 shows a visual representation of the states and transitions of the DTMC model.

Just by looking at transitions and their weight (the values associated to the modelled probabilities decorating transitions), one might assume that the expected probability result for both `Rainy` and `Cloudy` states will be high. Seldom it transitions to the `Sunny` state, and from this state, it returns with high probability towards `Rainy` (0.5) or `Cloudy` (0.3) states.

The solution of this model (`spreadsheets/Chapter02-DTMC-3states.xlsx`) shows that the final probability vector π is distributed as: $\pi_{\texttt{Rainy}} = 0.7625$, $\pi_{\texttt{Cloudy}} = 0.16875$, $\pi_{\texttt{Sunny}} = 0.06875$. So, `Rainy` or `Cloudy` days account for almost the totality of probabilities, `Rainy` + `Cloudy` $= 0.7625 + 0.16875 = 0.93125$, which is the same of thinking `Rainy` + `Cloudy` $= 1 -$ `Sunny` (for this model), yielding the same result. So, one could expect that any given day, in Belfast, to be either `Rainy` or `Cloudy`, and prepare for this.

2.3 Let's code!

This challenge will work with static structures and simple tests to see whether the model is well-written as expected for MC. You will need to review ergodicity, absorbing states, and the previously discussed properties of well-formed MC for this challenge.

[1] Häggström [2] contrasts two other weather models, with comparisons, namely the *Gothenburg* model and the *Los Angeles* model.

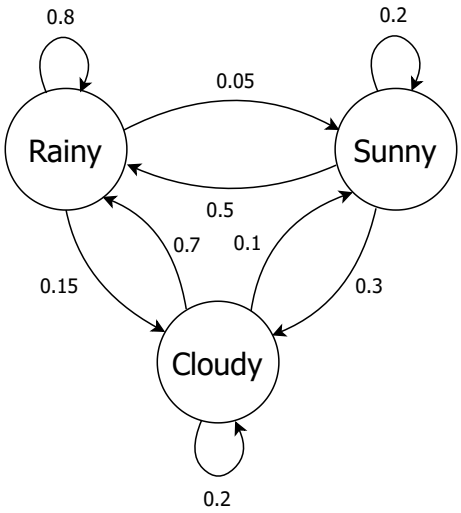


Figure 2.2: Model representing the weather of Belfast, in Northern Ireland.

2.3.1 Challenge 03

Before we start discussing the challenge, it is worth noticing that we are doing *validation* instead of *verification*[2].

File: `challenge03.c`

Do: write a program that declares a static matrix and runs some tests (e.g., sum of lines equals to one, etc.) and output features.

Notes:

- The matrix corresponding to the model could be defined hard coded with static declarations (`float m[3][3];`), however, you could implement a way of opening a (text) file with the model (where values are separated by spaces or commas). That would require dynamic allocation and pointers.

Good programming practice:

- Use `#define LIN 3` and `#define COL 3`.
- Employ error codes (as `#define`) for returning values in the function, for example, `ERR_NOT_SQUARE`, `ERR_ABSORBING`,

[2]The word *verification* conjures the notion of *formal verification* processes that models might undergo against specifications.

`ERR_LINE_SUM`, `SUCCESS`).

Implement:

- Function `void print_static_dtmc(float m[LIN][COL]);`
- Function `void print_static_array(float a[LIN]);`
- Function `int validate_static_dtmc(float m[LIN][COL]);` that checks whether every line of matrix m sums 1.0 and also if the matrix m has absorbing states.
  - Return 1 if matrix checks out and it is well-formed, no absorbing, summing lines to 1.0.
  - Return -1 if the matrix is not square (`#lines` ! = `#columns`).
  - Return -2 if sum of each line (show line) is not 1.0.
  - Return -3 if matrix has absorbing states: check whether there is one state with no outgoing transitions to other states other than to itself.

Tips:

- Look at each state's *outgoing* transitions: state i is absorbing if $m_{ii} = 1$, i.e., every other entry in row i is zero, so once the chain enters i it never leaves. Incoming transitions do not matter: a state with no incoming transitions is merely unreachable (transient), which breaks irreducibility (Section 1.9.1) but is a different problem.
  - In code, this is a single test per row: `if (fabsf(m[i][i] - 1.0f) < 1e-6f)`, then state `i` is absorbing.
    - Use a small tolerance rather than `==`, because values read from a file may not be exactly `1.0`.
    - `fabsf` needs `#include <math.h>`, so compile with `-lm` at the end, e.g., `gcc -Wall -o challenge03 challenge03.c -lm`.
- Implement a way to learn about the state which is absorbing (pass a pointer to an integer by reference to the `validate_static_dtmc` function, where it saves it to this variable). Then show the state in the output.

Suggested variations: (no code or auxiliary file will be provided)

- 'Open' the model from a file, and then 'Save' the output to a file.
- Use a library (external API) to power a matrix or multiply a vector by a matrix.
- Run the model on MATLAB® or GNU/Octave and compute the same set of results.

2.4 Solution methods for DTMC

After the model definition and after passing basic tests to see whether it is well-formed and no absorbing states (which would require other solution methods), it is time to solve it and extract state (occurrence) probabilities.

We will work next with three methods: i) Power matrix; ii) VMM; and iii) forward simulation. For i) and ii) methods, the idea is to call the functions iteratively until one achieves convergence, i.e., the resulting matrix has the same vectors in all positions, or the resulting vector does not change between the last two iterations. If the solution reaches the boundaries set by the numerical methods (e.g., number of iterations), it then diverged (the opposite of converged), and you should state this to whomever call the method and show the last computed vector.

2.4.1 Power matrix, or Matrix-Matrix Multiplication

This method consists of multiplying a matrix (in our case, one where each line sums to 1.0) by itself, iteratively, until one observes negligible differences (or no difference at all) between lines in the final matrix. Because this method employs ensembles of multiplication/addition pairs each time, depending on the size of the model, it may behave computationally heavy in terms of required processing power.

$$\begin{pmatrix} a_{00} & a_{01} & a_{02} \\ a_{10} & a_{11} & a_{12} \\ a_{20} & a_{21} & a_{22} \end{pmatrix}$$

Note that there are no convergence insurances that this method will produce valid π probabilities vector after execution. It is the job of the modeller to analyse (partial) results and investigate causes when diverging, refining models, reviewing transition probabilities, and so on.

2.4.2 Vector-Matrix Multiplication

The VMM method will operate in simpler terms, it will multiply an initial vector (where the sum is 1.0) by the matrix iteratively, until this vector has negligible differences between two iterations.

$$\begin{pmatrix} 1 & 0 & 0 & 0 \end{pmatrix} \times \begin{pmatrix} a_{00} & a_{01} & a_{02} & a_{03} \\ a_{10} & a_{11} & a_{12} & a_{13} \\ a_{20} & a_{21} & a_{22} & a_{23} \\ a_{30} & a_{31} & a_{32} & a_{33} \end{pmatrix}$$

The power matrix method may converge faster than this method, however, VMM requires less mathematical operations to complete on each iteration. On each iteration of VMM it is possible to see the π vector being redistributed throughout the positions, always summing to 1.0 (as a matter of fact it will sum to the value set earlier).

Let's revisit the Belfast weather model and apply the VMM method:

$$\begin{pmatrix}1 & 0 & 0\end{pmatrix} \times \begin{pmatrix}0.80 & 0.15 & 0.05\\ 0.70 & 0.20 & 0.10\\ 0.50 & 0.30 & 0.20\end{pmatrix}$$

Here is a step-by-step view of the VMM process.

Consult file `spreadsheets/Chapter02-DTMC-3states.xlsx`, on Tab 'Vector-Matrix Multiplication':

```
Initial Vector: 1 0 0
Iter.1: 0.8         0.15        0.05
Iter.2: 0.77        0.165       0.065
Iter.3: 0.764       0.168       0.068
Iter.4: 0.7628      0.1686      0.0686
Iter.5: 0.76256     0.16872     0.06872
Iter.6: 0.762512    0.168744    0.068744
Iter.7: 0.7625024   0.1687488   0.0687488
Iter.8: 0.76250048  0.16874976  0.06874976
Iter.9: 0.762500096 0.168749952 0.068749952
```

Note that by Iteration 9, the residue between Iteration 8 and this one is negligible. Depending on accuracy requirements, this could be handled by continuing the process until the residue is smaller than a pre-defined threshold.

2.4.3 Forward simulation

For the DTMC simulation (also called *forward* simulation), the process is different, and one must follow this process:

- Define the desired number of visits or samples (e.g., 1000).
- Define a start state to visit initially (any state belonging to the model).
- Modify the probability matrix of the DTMC to withhold the cumulative probabilities on each position, that will ease the selection of the next state.

For example, suppose a three state DTMC with the following probabilities:

$$\begin{array}{c} \\ \mathbf{0} \\ \mathbf{1} \\ \mathbf{2} \end{array} \begin{array}{c} \begin{array}{ccc}\mathbf{0} & \mathbf{1} & \mathbf{2}\end{array} \\ \begin{pmatrix}0.2 & 0.4 & 0.4\\ 0.1 & 0.2 & 0.7\\ 0.3 & 0.6 & 0.1\end{pmatrix} \end{array} \quad \texttt{becomes} \quad \begin{array}{c} \\ \mathbf{0} \\ \mathbf{1} \\ \mathbf{2} \end{array} \begin{array}{c} \begin{array}{ccc}\mathbf{0} & \mathbf{1} & \mathbf{2}\end{array} \\ \begin{pmatrix}0.2 & 0.6 & 1.0\\ 0.1 & 0.3 & 1.0\\ 0.3 & 0.9 & 1.0\end{pmatrix} \end{array}$$

On the left-hand side, there is the original matrix, and on the right-hand side, an accumulated version, on each state, of the same matrix.

When making draws for each state, let's say in state 0, and a pseudo-random number equals to 0.345632, then it suffices to consult the accumulated table and see

that if the number is in $[0, 0.2)$ the next state is 0, if it is in $[0.2, 0.6)$ the next state is 1, otherwise ($[0.6, 1)$) it is state 2. For 0.345632, the next state is therefore 1.

Let's continue the process. From that chosen start state, one:

1. Check if the number of samples reached the desired value: if yes, stop the process and go directly to step 6;
2. Add the visit to this state in a counting vector holding all visits;
3. Draw a uniform random number r;
4. In the current state's row of the cumulative matrix, find the first column whose value is greater than r, and jump to that state;
5. Repeat steps 1–4;
6. Compute visit statistics (e.g., divide each position of the counting vector by the total number of samples);
7. Present the vector to the user (it should approximate its analytical counterpart).

Let's proceed now to a demonstration of this procedure.

Consult `spreadsheets/Chapter02-DTMC-3states.xlsx`, Tab 'DTMC simulator'. It shows the accumulated matrix and $1,000$ samples from the Belfast weather model (note that the results approximate to the analytical answer).

2.4.4 Useful comments

Note that sometimes running a method and inspecting the resulting probability vector may help you investigate other problems with models. Depending on the problem, one could resort on modifying the original matrix to *force its way* into becoming a MC, i.e., adding transitions or even new states (that is the approach employed by Google's PageRank® when (attempting to) *fixing* the matrix).

One clear indication of problems is, for example, if just one state has all the probability – this means that your model has an absorbing state If only a small set of states have probabilities, then one should inspect whether the MC is irreducible or perhaps NCD (with weak links).

2.4.5 Direct solution method

It is worth discussing that these three solution methods are not the only way of solving MC. For instance, one might employ direct solution methods (linear equations) and specialised tools. We shall briefly comment on these two methods next.

The direct solution methods employ for example MATLAB® or GNU/Octave. The idea is to work with Equation 2.3 [11, 3]:

$$\pi \times P = \pi \tag{2.3}$$

Recall that π is a vector of unknowns, i.e., for the Belfast weather model (with three states, according to Section 2.2 with a probability vector having size three: $[\pi_{\texttt{Rainy}}, \pi_{\texttt{Cloudy}}, \pi_{\texttt{Sunny}}]$). This is what we would like to compute, i.e., the probability of each state in the model. To simplify the discussion, we shall refer this symbolic probability vector as $[\pi_1, \pi_2, \pi_3]$.

If you recall the Belfast weather model, here are the state transitions and the required operations for performing $\pi \times P = \pi$:

$$\begin{bmatrix}\pi_1 & \pi_2 & \pi_3\end{bmatrix} \times \begin{pmatrix}0.80 & 0.15 & 0.05\\ 0.70 & 0.20 & 0.10\\ 0.50 & 0.30 & 0.20\end{pmatrix} = \begin{bmatrix}\pi_1\\ \pi_2\\ \pi_3\end{bmatrix}$$

Refer to the review in matrix multiplication we conducted in Section 2.1.
Let's multiply this symbolically:

$$\begin{cases}0.80\pi_1 + 0.70\pi_2 + 0.50\pi_3 &= \pi_1\\ 0.15\pi_1 + 0.20\pi_2 + 0.30\pi_3 &= \pi_2\\ 0.05\pi_1 + 0.10\pi_2 + 0.20\pi_3 &= \pi_3\end{cases}$$

We should work with the variables and also add the following equation to the mix: $\pi_1 + \pi_2 + \pi_3 = 1$. So, after operating (after some algebra) on the equations, we come up with the following system to solve:

$$\begin{cases}(0.8-1)\pi_1 + 0.70\pi_2 + 0.50\pi_3 &= 0\\ 0.15\pi_1 + (0.2-1)\pi_2 + 0.30\pi_3 &= 0\\ 0.05\pi_1 + 0.10\pi_2 + (0.2-1)\pi_3 &= 0\\ 1.00\pi_1 + 1.00\pi_2 + 1.00\pi_3 &= 1\end{cases} \quad \begin{cases}-0.20\pi_1 + 0.70\pi_2 + 0.50\pi_3 &= 0\\ 0.15\pi_1 - 0.80\pi_2 + 0.30\pi_3 &= 0\\ 0.05\pi_1 + 0.10\pi_2 - 0.80\pi_3 &= 0\\ 1.00\pi_1 + 1.00\pi_2 + 1.00\pi_3 &= 1\end{cases}$$

This is a system of linear equations with three unknowns. The best (and fastest) way to solve this is using MATLAB® (or GNU/Octave) for discovering the π probability vector (refer to file `matlab/matlabsolution_dtmc.m` in the GitHub repository).

```
% MATLAB Model
M = [ -0.20,  0.70,  0.50;
       0.15, -0.80,  0.30;
       0.05,  0.10, -0.80;
       1.00,  1.00,  1.00 ];
b = [0; 0; 0; 1];
pi = linsolve(M,b);
```

The solution vector is: $[\pi_{\text{Rainy}}, \pi_{\text{Cloudy}}, \pi_{\text{Sunny}}] = [0.7625, 0.1688, 0.0688]$ which (not surprisingly) corresponds to the solution for this model (using the Power method or VMM as explained earlier).

2.4.6 PRISM DTMC model

The PRISM model is straightforward, being sufficient to describe the model using the appropriated syntax, i.e.:

```
dtmc

const int N=3; // N - number of states

const int sRainy  = 0;
const int sCloudy = 1;
const int sSunny  = 2;

module Module1
 n : [0..N] init sRainy; // total states, initial state sRainy
 // transitions
 [] (n=sRainy)  -> 0.80 : (n'=sRainy)  +
                   0.15 : (n'=sCloudy) +
                   0.05 : (n'=sSunny);
 [] (n=sCloudy) -> 0.20 : (n'=sCloudy) +
                   0.70 : (n'=sRainy)  +
                   0.10 : (n'=sSunny);
 [] (n=sSunny)  -> 0.20 : (n'=sSunny)  +
                   0.50 : (n'=sRainy)  +
                   0.30 : (n'=sCloudy);
endmodule
```

The model is located in `prism/belfast-dtmc.pm` and it should be run in PRISM Probabilistic Model Checker tool (it was tested on version 4.6). After running the tool, go to `Open Model` and choose a file. Click on `Model>Compute>Steady-state probabilities`. The following results are shown to modellers:

```
Printing steady-state probabilities in plain text format below:
0:(0)=0.7624998618647714
1:(1)=0.16875010721113942
2:(2)=0.06875003092408917
```

Observe that it yielded the same probability vector as output as all other methods explored here so far.

For the next challenge it will be required to work with C pointers and dynamic allocation so you should sharpen up your skills around these concepts before venturing into the challenge.

2.5 Let's code!

The next programming challenge will take into account those aforementioned matrix and vector operations.

2.5.1 Challenge 04

We will implement a solution to work with the power method and the VMM.

File: `challenge04.c`

Do: write a program to work with the following methods:

1. Power matrix
2. Vector-Matrix Multiplication

Notes:

- We will need to use dynamic allocation (employing pointers) in C for the next tasks.

Good programming practice:

- When programming with pointers you need to exert caution and patience.
- Your code should not run indefinitely. You will have to code auxiliary functions to help you, for example, one that checks residuals between two arrays of floats. If the residuals are less than a pre-defined threshold you may stop computing the process.

Implement a program accepting the following parameters:

- `<FILENAME> <OP>`
  - <FILENAME>: a text-based file with a DTMC (a squared matrix, each line sums 1.0) describing the model.
  - OP (will trigger one of the following methods):
    - [OP=0] `void multiply0(float** m, int size);`
      - Squares the matrix `m` (of order `size`) in place; call it repeatedly until all rows are equal, which gives the solution vector.
    - [OP=1] `void multiply1(float* v, float** m, int size);`
      - Multiplies the vector `v` by `m` in place (one VMM step); call it repeatedly until `v` stops changing.
  - The usual tests (see Section 2.3.1) apply and at the end, it prints the full resulting matrix (for $OP = 0$), the resulting vector (for $OP = 1$).

Suggested variations:

- Output the current result vector per iteration on the screen (according to an option passed in command-line).

2.5.2 Challenge 05

This challenge will implement a basic forward DTMC simulator.

File: `challenge05.c`

Do: write a program that runs a simple DTMC simulator, where given a model, a visitor jumps from state to state during a pre-defined number of steps.

Notes:

- Implement a function `void dtmc_simulator(float** m, int size, int samples, int* results);` that simulates a DTMC (it stores the number of visits to each state in `results`; dividing by `samples` gives the estimate of π).
- The results should approximate to the analytical solution as observed in previous challenge (Section 2.5.1).

Implement a program accepting the following parameters:

- `<FILENAME> <SAMPLES>`

Implementation tips:

- Not necessarily you will need to code the accumulating matrix. You could just use an accumulating variable each time you traverse the line of the matrix under consideration (this will be explored in the solution for this challenge).

Suggested variations:

- Output the top N states (passed in command line) with the highest visits (so far) as the method is running).

2.6 Comments on the spreadsheets

The spreadsheets for DTMC are:

- The Lighting model: `spreadsheets/Chapter02-DTMC-2states.xlsx`.
- The Belfast model: `spreadsheets/Chapter02-DTMC-3states.xlsx`.

Each file has three tabs: 1. Power-Matrix method; 2. Vector-Matrix Multiplication; and 3. DTMC simulator.

The first method repeatedly squares the model's matrix ($M^2, M^4, \ldots$; up to M^{64} for the Lighting model), whereas the VMM method iteratively multiplies a start vector

(summing to one) by the model's matrix until the last two resulting vectors are the same between two iterations (in this case, it ran for nine iterations).

Finally, the DTMC simulator will run a simulation using pseudo-random numbers (uniformly distributed) mimicking the idea of visiting states one after the other for a fixed number of times (in the spreadsheet, it is set to run until it produces 1,000 state samples).

After it runs, it computes the resulting probabilities. In terms of results, one notices that they are quite close to the analytical ones (computed previously), however, a larger number of samples would produce even closer results.

Chapter 3

CTMC

In terms of modelling, one could wonder about the expressiveness of CTMC over DTMC. The idea is that one could think about state permanence, or durations, instead of probabilities. CTMC are different than DTMC when modelling situations because one thinks on the time spent *in* the state (residence or sojourn time) and how it transitions to other states, using $\texttt{rate} = \frac{1}{duration}$ to represent the rate *out* of the state.

Figure 3.1 shows how the balancing process works for each state: the negative sum of outgoing rates is used to even out the rates, so each line in the model will sum to zero.

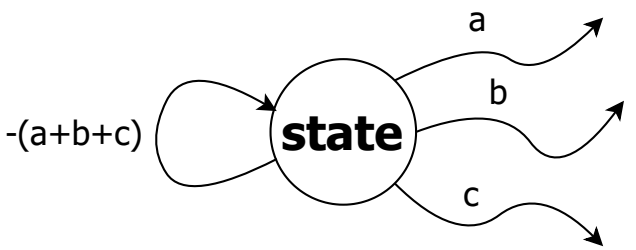


Figure 3.1: Representation of balancing rates in states in CTMC.

Here, this is not a MC *per se*, but only a representation of how and why one does not consider self-loop transitions. The side-effect of this is that, in CTMC, the diagonal is not used in the modelling process, because it is destroyed to accommodate the sum of exits and balance out the rates in and out the state.

3.1 Infinitesimal Generator

The process to model CTMC differs from DTMC in the sense that we must work with a special kind of matrix called the Infinitesimal Generator (IG). It consists on

destroying the diagonal (self-loops in the model) and substituting it with the negative sum of rates of this state, balancing *rates in* and *rates out*.

Let's revisit the Lighting model once again in Figure 3.2:

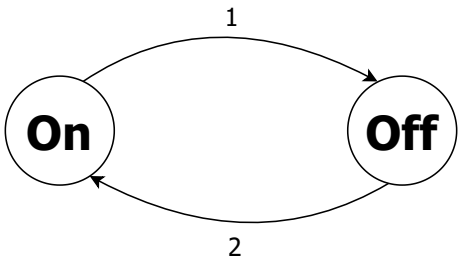


Figure 3.2: Lighting model with two states as a CTMC.

The IG (referred as matrix Q) for this model is[1]:

$$\mathrm{Q} = \begin{pmatrix} -1 & 1 \\ 2 & -2 \end{pmatrix}$$

The same process is repeated for higher order matrices, i.e., a Generic model with four states (Figure 3.3):

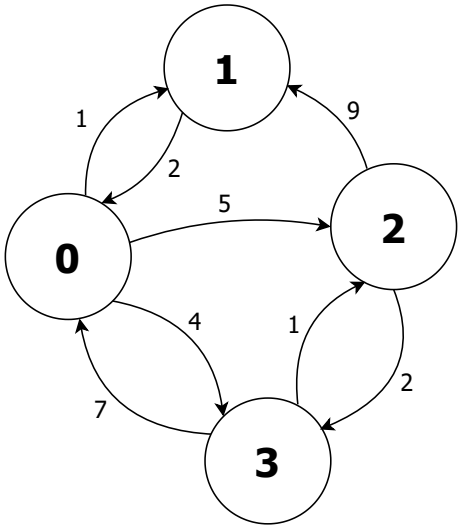


Figure 3.3: Generic CTMC model with four states.

[1]It contrasts with matrix M in DTMC, where each line sums 1.0, whereas Q sums 0.0.

The corresponding IG for this model is:

$$\mathrm{Q} = \begin{pmatrix} -10 & 1 & 5 & 4 \\ 2 & -2 & 0 & 0 \\ 0 & 9 & -11 & 2 \\ 7 & 0 & 1 & -8 \end{pmatrix}$$

Note that both models respect previously mentioned properties to be MC, i.e., they are ergodic (Section 1.9).

3.2 Computing the uniformised DTMC

Given an IG, one can convert the CTMC into its uniformised DTMC. The process is as follows.

1. Let Q be an Infinitesimal Generator of a MC model.
2. Let I be the Identity Matrix of the same order as Q.
3. Find the uniformisation constant Λ, i.e., the largest exit rate of any state:

$$\Lambda = \max_i |q_{ii}|. \tag{3.1}$$

   Because each diagonal entry is minus the sum of that row's rates, Λ is the largest absolute value on the diagonal. Any larger value would also work.
4. Compute

$$M' = I + \frac{Q}{\Lambda}, \qquad \text{i.e.,} \qquad M'_{ij} = I_{ij} + \frac{q_{ij}}{\Lambda}. \tag{3.2}$$

Let us apply this to the Generic model from the previous section. Its diagonal is $-10, -2, -11, -8$, so $\Lambda = 11$:

$$\underbrace{\begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix}}_{I} + \frac{1}{11} \underbrace{\begin{pmatrix} -10 & 1 & 5 & 4 \\ 2 & -2 & 0 & 0 \\ 0 & 9 & -11 & 2 \\ 7 & 0 & 1 & -8 \end{pmatrix}}_{Q} = \underbrace{\begin{pmatrix} \frac{1}{11} & \frac{1}{11} & \frac{5}{11} & \frac{4}{11} \\ \frac{2}{11} & \frac{9}{11} & 0 & 0 \\ 0 & \frac{9}{11} & 0 & \frac{2}{11} \\ \frac{7}{11} & 0 & \frac{1}{11} & \frac{3}{11} \end{pmatrix}}_{M'}$$

which, numerically, is

$$M' \approx \begin{pmatrix} 0.09 & 0.09 & 0.45 & 0.36 \\ 0.18 & 0.82 & 0.00 & 0.00 \\ 0.00 & 0.82 & 0.00 & 0.18 \\ 0.64 & 0.00 & 0.09 & 0.27 \end{pmatrix}.$$

For example, the first cell is $M'_{00} = 1 + \frac{-10}{11} = \frac{1}{11} \approx 0.0909$; off-diagonal cells are simply $q_{ij}/11$.

M' is the uniformised DTMC of Q: each row sums to one, and its stationary vector is the CTMC's π, so any of the DTMC solution methods discussed earlier can be applied to it directly.

Observe that M' has self-loops (from 0 to 0, 1 to 1, and 3 to 3). They appear because uniformisation advances the chain in ticks of a single rate Λ: a state whose exit rate is lower than Λ has nothing to do on some ticks, so it stays put. Only the state with the largest exit rate (state 2) has no self-loop.

3.2.1 Computing the embedded DTMC

The embedded DTMC separates *where* the process goes next from *how long* it stays. Its matrix P holds only the jump probabilities, with a zero diagonal:

$$P_{ij} = \begin{cases} \dfrac{q_{ij}}{|q_{ii}|}, & i \neq j, \\ 0, & i = j, \end{cases} \qquad \text{where } |q_{ii}| = -q_{ii} = \sum_{k \neq i} q_{ik}, \tag{3.3}$$

while the sojourn time in state i is exponential with rate $|q_{ii}|$.

For the Generic model, dividing each row of Q by the absolute value of its diagonal gives:

$$Q = \begin{pmatrix} -10 & 1 & 5 & 4 \\ 2 & -2 & 0 & 0 \\ 0 & 9 & -11 & 2 \\ 7 & 0 & 1 & -8 \end{pmatrix} \implies P = \begin{pmatrix} 0 & 0.10 & 0.50 & 0.40 \\ 1.00 & 0 & 0 & 0 \\ 0 & 0.82 & 0 & 0.18 \\ 0.875 & 0 & 0.125 & 0 \end{pmatrix}$$

Compare it with M' in Section 3.2: there are no self-loops.

The stationary vector ν of P ($\nu P = \nu$) counts *visits*, not time. To recover the CTMC probabilities π, weight each state by its mean sojourn time $1/|q_{ii}|$ and normalise:

$$\pi_i = \frac{\nu_i / |q_{ii}|}{\sum_k \nu_k / |q_{kk}|}. \tag{3.4}$$

For the Generic model, $\nu = [0.3809,\ 0.2135,\ 0.2143,\ 0.1913]$, which Equation 3.4 turns into $\pi = [0.2024,\ 0.5671,\ 0.1035,\ 0.1271]$, the same result as the other methods.

Building the embedded DTMC has three practical advantages:

1. It gives a simpler and faster CTMC simulator: rather than drawing one exponential per outgoing transition and picking the smallest (Section 3.6), one draws a single uniform number to choose the next state from row i of P, and a single exponential for the time spent in i.
2. P has no self-loops and does not depend on the uniformisation constant $\Lambda = \max_i |q_{ii}|$ (Section 3.2), so it reflects the actual sequence of states visited. That sequence is what matters for questions such as hitting or absorption probabilities and the expected number of visits before an event.

3. It is numerically better behaved when rates differ by orders of magnitude. With uniformisation, slow states get self-loops close to 1, and iterative methods converge slowly; the embedded chain avoids this. The price is the reweighting step of Equation 3.4.

3.3 The Lily Pad model

Let's consider the "Lily Pad model" now: it consists of a frog hopping lily pads in a fictitious pond. The idea is that it jumps around mindlessly, staying some time on each lily pad and then moving on to the next.

Figure 3.4 shows a pond with four lily pads and transitions among them.

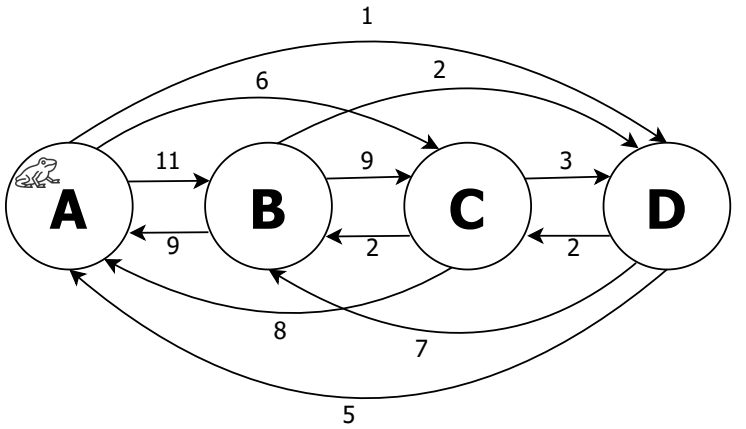


Figure 3.4: Lily Pad model showing possible pads and transitioning rates.

Consider Table 3.1 for thinking about the rates decorating transitions in the model. Note that for modelling time, the observer has attended the pond and remained collecting frog residence times over a period of time, to have a nice idea of jumps and durations.

One might think that, when modelling, it is easier to consider states and 'entities' changing states where each has remained some time in the state before transitioning to other state. The reasoning is that sometimes, time is the only observable thing to model the system. Recall that for modelling DTMC one requires the probabilities for switching states. Because of the formula to convert durations to rates, lower time spent in the state corresponds to higher rates out of it, as seen in the table. For example, from pad A to pad B with a duration of 5.4 min, the converted rate was equal to 11, whereas from pad A to pad D, the frog stayed for 60 min and the rate out in one hour was one.

The IG (Q) for this model ($\Lambda = 20$) and its uniformised DTMC M' is:

Table 3.1: Observed time until each transition from each lily pad, and the corresponding rates.

Time (min)	From	To	Rate
5.4	A	B	11
10		C	6
60		D	1
6.7	B	A	9
6.7		C	9
30		D	2
7.5	C	A	8
30		B	2
20		D	3
12	D	A	5
8.6		B	7
30		C	2

$$\mathtt{Q} = \begin{pmatrix} -18 & 11 & 6 & 1 \\ 9 & -20 & 9 & 2 \\ 8 & 2 & -13 & 3 \\ 5 & 7 & 2 & -14 \end{pmatrix} \qquad \mathtt{M'} = \begin{pmatrix} 0.10 & 0.55 & 0.30 & 0.05 \\ 0.45 & 0.00 & 0.45 & 0.10 \\ 0.40 & 0.10 & 0.35 & 0.15 \\ 0.25 & 0.35 & 0.10 & 0.30 \end{pmatrix}$$

As previously mentioned, with the uniformised DTMC M', one could employ Power matrix, VMM, simulation, or direct methods to discover π. Using the VMM, we reach the following results (starting with vector `[ 1 0 0 0 ]`):

$$\pi = \begin{pmatrix} 0.302449696 & 0.243404608 & 0.327575347 & 0.12657035 \end{pmatrix}$$

We come to the conclusion that the frog has a preference for lily pads A, B, and C ($1 - 0.12657035 = 0.87342965$, $\approx$87% of the time), given the probabilities that we have computed. It is worth mentioning that those are the same results if one employed the Power matrix method using the uniformised DTMC.

3.4 Let's code!

Let's put those ideas into practice now.

3.4.1 Challenge 06

This challenge 'fix' the diagonal of a CTMC and then convert it to its uniformised DTMC representation.

File: `challenge06.c`

Do: write a program that will take a CTMC as parameter and then compute the main diagonal accordingly and convert it to its uniformised DTMC.

Notes:

- Implement a function `void fix_ctmc(float** q, int size);` that will destroy the current main diagonal and replace it with the sum of line accordingly.
- Implement a function `float** convert2dtmc(float** q, int size);` that creates another matrix in memory and returns the DTMC of the CTMC matrix passed by parameter (variable q)/
  - Remember to use previous function `fix_ctmc` because it has the 'corrected' value for the diagonal.
- Remember that you will have to code auxiliary functions to help you, such as `float discover_lambda(float** q, int size)`, which returns the uniformisation constant $\Lambda = \max_i |q_{ii}|$ (Equation 3.1).
- You will also need to consider the use of *identity matrix* (without necessarily creating it in memory.
  - The idea is that if you are traversing Q, then on the diagonal (when `i==j`) you compute $1 + q_{ii}/\Lambda$, otherwise q_{ij}/Λ.

Suggested variations:

- Remember to use previously implementations and solution mechanisms for DTMC (earlier challenges).
- Offer users a way of outputting the newly generated DTMC in a file (that you pass both the option and the name of this file in the command-line).

3.5 Solution methods for CTMC

3.5.1 Forward simulation

In the DTMC case, one would choose an initial state, and according to a model, visit states using the uniform distribution. The process ends when enough visits reached a pre-defined value, and then modellers would compute state statistics on visits.

For CTMC we would like to pursue more or less the same venue, however, now, we would like to generate pseudo-random numbers from the exponential distribution.

The idea of visiting states remains, also for running for a large number of samples and then calculating state probabilities.

Before we start discussing the simulation per se, we would like to state the process and how it would work, for example:

1. Given a model written in CTMC, and assuming that the diagonal contains the sum of the rest of the elements (negative value).
2. The model fills a matrix Q, that is a valid (well-written) Infinitesimal Generator.
3. Compute $Q' = Q + \Lambda I$, where $\Lambda = \max_i |q_{ii}|$ (Equation 3.1).
4. From an initial state S_0, draw a number from the exponential distribution (considering the rates in Q) and then visit the next state, accumulating all these visits in a counting vector (to present to modellers afterwards).
    - The formula to generate an exponential distributed variable using a pseudo-random number generated using the uniform distributions is $\texttt{Exp(rate)} = (-\frac{1}{\texttt{rate}}) \times log(1 - \text{U}(0;1), e)$, where $\text{U}(0,1)$ is a pseudo-random number between 0 and 1 and log is the natural logarithm of a number (base e).

Recall the Q matrix for the Generic model using CTMC (Section 3.2):

$$\text{Q} = \begin{pmatrix} -10 & 1 & 5 & 4 \\ 2 & -2 & 0 & 0 \\ 0 & 9 & -11 & 2 \\ 7 & 0 & 1 & -8 \end{pmatrix}$$

For reference, these are all intermediary computations for computing $Q' = Q + \Lambda I$ (with $\Lambda = 11$); only the diagonal changes:

$$Q' = \begin{pmatrix} -10+11 & 1 & 5 & 4 \\ 2 & -2+11 & 0 & 0 \\ 0 & 9 & -11+11 & 2 \\ 7 & 0 & 1 & -8+11 \end{pmatrix} = \begin{pmatrix} 1 & 1 & 5 & 4 \\ 2 & 9 & 0 & 0 \\ 0 & 9 & 0 & 2 \\ 7 & 0 & 1 & 3 \end{pmatrix} = \Lambda M'.$$

If you simulate this matrix, you will reach results that are approximations of the other solution methods for this model.

3.6 Race condition

When modelling CTMC and looking at individual states, one might ponder: if one must choose a state to visit, given the rates, which one to select? This is important in simulation, because visiting a state one cannot look at the past (after all this is a Markovian process) to decide which state to visit next.

Look at Figure 3.5 next, given that you are in a state with those transitions, which one you choose to trigger next?

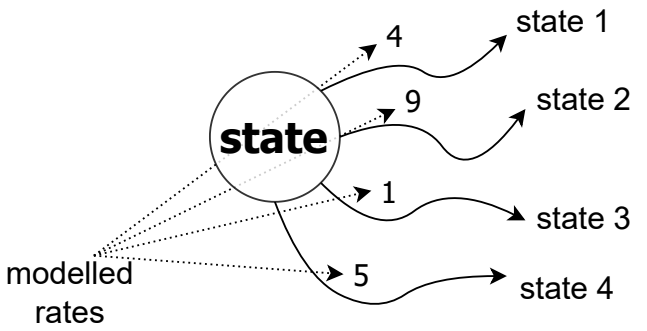


Figure 3.5: Hypothetical situation showing the problem of choosing the next state in CTMC simulation.

The answer is: the one that fires first. Each outgoing transition is an exponential "clock" with its own rate; one draws a time from each clock, and the smallest time wins the race. The rates still matter, because a higher rate tends to give shorter times, so it wins more often. For a state with outgoing rates q_{ij}, transition j wins with probability

$$\Pr(j \text{ wins}) = \frac{q_{ij}}{\sum_{k \neq i} q_{ik}} = \frac{q_{ij}}{|q_{ii}|}, \tag{3.5}$$

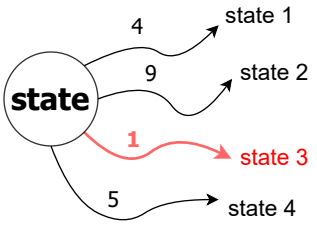
and the winning time, which is how long the process stays in state i, is itself exponential with rate $|q_{ii}|$. For Figure 3.5 the total rate is $4 + 9 + 1 + 5 = 19$, so state 2 wins about $9/19 \approx 47\%$ of the time and state 3 only $1/19 \approx 5\%$.

Important Remember that Figure 3.5 does not show previous transitions for reaching the state: recall that in MC the past is *irrelevant*, what only matters is where to 'jump' next.

In a CTMC simulation you are working with time; so, this means that you will draw pseudo-random numbers using each state rates and then you will choose the one that happened first in time. Figure 3.6 details this process, and shows the winning transition for this particular case.

So, given the pseudo-random numbers, and after establishing a rank (to simulate the lowest time), one decides that `state 3` will be the next state visited[2]. This is the core of the CTMC simulator, visiting states, and depending on the rates on each state, deciding where to go next, simulating time. Since you are simulating, you could choose how long would you like to simulate in terms of time, e.g., 10 hours.

[2]State 3 has the lowest rate, so it wins only about 5% of the time (Equation 3.5). It won here by chance. Repeat the draws in a spreadsheet and count the winners: the proportions will approach $4/19$, $9/19$, $1/19$ and $5/19$.



Rate	Uniform	Exponential	Rank
4	0.3024550	0.0900470	2
9	0.8295417	0.1965850	4
1	0.0528990	0.0543496	**1**
5	0.6072475	0.1869151	3

$$\texttt{Exp(rate)} = \left(\frac{-1}{rate}\right) \times log(1 - \texttt{Unif(0;1)}, e)$$

Figure 3.6: Drawing pseudo-numbers from uniform and exponential distributions to help simulate time in CTMC (`log` is the natural logarithm, in base e).

Equation 3.5 is exactly row i of the embedded DTMC (Section 3.2.1), so racing the clocks and drawing once from P give the same simulator: the race costs one exponential per transition, the embedded chain only one uniform and one exponential per step.

3.7 Comments on the spreadsheets

This are the MS-Excel files detailing the solution possibilities for this model:

- Generic model: `spreadsheets/Chapter03-CTMC-4states-generic.xlsx`.
- Lily Pad model: `spreadsheets/Chapter03-CTMC-4states-lily.xlsx`.

3.8 Let's code!

For this challenge we will build a CTMC simulator.

3.8.1 Challenge 07

File: `challenge07.c`

Do: write a program that will take a CTMC as parameter and then simulates it for some time (passed as parameter in the command line).

Implement a program accepting the following parameters:

- `<FILENAME> <TIME>`

Notes:

- Implement a function `float** convert2ctmc_prime(float** q, int size);` that converts a matrix Q by another (Q') using the method

discussed in Section 3.5.1.

- Use the function that generates a pseudo-number from the exponential distribution that you have coded in previous challenge (Challenge 02 in Section 1.8.2) – the function needs the `rate` parameter.
  - Function prototype: `float exponential(float n);`
- You will need to implement a function to find out the next state to visit. The idea is to draw N exponentially distributed numbers (a line of the current state) and then select the one that has the least value other than zero, so code the following function:
  - `int lowest_not_zero(float* qline, int size, float* sim_time);`
    - It returns the next state according to values drawn
    - You must pass the time in order to conveniently update it (since the function returns the next state).
- Implement a function `int ctmc_simulator(float** qprime, int size, float sim_time, int* results);` that simulates a CTMC (it stores visit counts in `results` and returns the number of samples; dividing gives the estimate of π) for some time, i.e., `sim_time`.
  - The function returns the number of samples produced for simulated that time amount (units of time).

Suggested variations:

- Make the residue either small enough (configured in command line) or after sampling a number of samples (as before), whichever event comes early. Show each position's residue after every $1,000$ samples computation.
- Build a program that computes series of simulation 'batches' consisting of N samples each (for instance, 10 runs of 1000 samples), then present to the modeller basic statistics over the batches, e.g., averages, standard deviation, boxplot output (1st quarter, 2nd, 3rd, 4th). Look at Jain's book [14] for multiple batch runs.
- Implement transitions having other probability distributions (you are running a simulation after all).

There is a CTMC simulator written for MS-Excel in the following files (on Tab CTMC simulator):

- Lighting model: `spreadsheets/Chapter03-CTMC-2states.xlsx`.

- Generic model: `spreadsheets/Chapter03-CTMC-4states-generic.xlsx`.
- Lily Pad model: `spreadsheets/Chapter03-CTMC-4states-lily.xlsx`.

The analytical methods (Power matrix, VMM, direct solution, PRISM) yield the same results for each model; the simulators only approximate them, and get closer as the number of samples (or the simulated time) grows.

3.9 Solution methods (cont.)

Now let's continue working with other solution methods for CTMC, more precisely, the direct solution method and then some PRISM models.

3.9.1 Direct solution method

For direct solution of CTMC one must satisfy $\pi Q = 0$. Let's recall the Generic model (CTMC) and state transitions and add the required operations accordingly:

$$\begin{bmatrix}\pi_1 & \pi_2 & \pi_3 & \pi_4\end{bmatrix} \times \begin{pmatrix} -10 & 1 & 5 & 4 \\ 2 & -2 & 0 & 0 \\ 0 & 9 & -11 & 2 \\ 7 & 0 & 1 & -8 \end{pmatrix} = \begin{bmatrix}0\\0\\0\\0\end{bmatrix}$$

After multiplication, one has the following system of equations (plus the one for ensuring that all π will sum to 1.0, i.e., $\pi_1 + \pi_2 + \pi_3 + \pi_4 = 1$):

$$\begin{cases} -10\pi_1 + 2\pi_2 + 7\pi_4 &= 0 \\ \pi_1 - 2\pi_2 + 9\pi_3 &= 0 \\ 5\pi_1 - 11\pi_3 + \pi_4 &= 0 \\ 4\pi_1 + 2\pi_3 - 8\pi_4 &= 0 \\ \pi_1 + \pi_2 + \pi_3 + \pi_4 &= 1 \end{cases}$$

For this model one could start by defining $\pi_1 = 1$ and then doing algebra to discover π_3 and π_4 (on the third and fourth equations), then circling back to π_2 (second equation). Then, the next step would be to **normalise** the results, i.e., divide each value by their sum so that all probabilities sum to 1.0.

We submitted this system of linear equations to MATLAB® (please, refer to file `matlab/generic_model_ctmc.m` in the GitHub repository).

```
% MATLAB Model
Q = [-10, 2, 0, 7;
1, -2, 9, 0;
5, 0, -11, 1;
```

```
4, 0, 2, -8;
1, 1, 1, 1];

% auxiliary vector with one position set as 1
b = [0; 0; 0; 0; 1];

pi = linsolve(Q,b);
```

Again, the computed vector π is equivalent to the other solutions.

3.9.2 PRISM CTMC models

We present next the Lily Pad model and the Generic model written in CTMC for PRISM. We start with the Lily Pad model (file `prism/lily-ctmc.sm`):

```
// Lily Pad model (frog in the pond)
ctmc

const int N=4; // N - number of states

const int sA=0;
const int sB=1;
const int sC=2;
const int sD=3;

module Module1
 n : [0..N] init sA; // total states, initial state sA
 // transitions
 [] (n=sA) -> 11 : (n'=sB) + 6 : (n'=sC) + 1 : (n'=sD);
 [] (n=sB) ->  2 : (n'=sD) + 9 : (n'=sC) + 9 : (n'=sA);
 [] (n=sC) ->  2 : (n'=sB) + 8 : (n'=sA) + 3 : (n'=sD);
 [] (n=sD) ->  5 : (n'=sA) + 7 : (n'=sB) + 2 : (n'=sC);
endmodule

------------------------
Printing steady-state probabilities in plain text format below:
0:(0)=0.30244974694693344
1:(1)=0.24340456372498753
2:(2)=0.32757532070253004
3:(3)=0.12657036862554888
```

And here is the Generic model (file `prism/generic-ctmc.sm`):

```
// Generic model
ctmc
```

```
const int N=4; // N - number of states

const int s0=0;
const int s1=1;
const int s2=2;
const int s3=3;

module Module1
 n : [0..N] init s0; // total states, initial state s0
 // transitions
 [] (n=s0) -> 1 : (n'=s1) + 5 : (n'=s2) + 4 : (n'=s3);
 [] (n=s1) -> 2 : (n'=s0);
 [] (n=s2) -> 9 : (n'=s1) + 2 : (n'=s3);
 [] (n=s3) -> 7 : (n'=s0) + 1 : (n'=s2);
endmodule

------------------------
Printing steady-state probabilities in plain text format below:
0:(0)=0.20235285239955195
1:(1)=0.567058916201716
2:(2)=0.1035294076461007
3:(3)=0.1270588237526314
```

Chapter 4

More projects and models

In this chapter we will comment about other projects and some models based on MC that are worth discussing.

4.1 Projects

The code present here and in the repository are far from having a *bug-free* status. As a starting project, one could improve the code and translate it to C++, or adding more controls, working with increased number of states[1].

4.1.1 Combine it all!

The idea here is to combine all pieces of code into one C project, putting functions in a different file for proper organisation. In the repository you may find a folder called `project01`, with a full project with all challenges explored here (combined in one place).

The 'services' provided by the project are the following command-line options (the executable is `markov`):

```
printf("This is the Markov chain solver [DD/MM/YYYY].\n");
printf("To run it, type: ./markov <FILE> <OP>\n");
printf(" where <OP> is one out of these options:\n");
printf("   0 Solve DTMC - Power Matrix\n");
printf("   1 Solve DTMC - Vector-Matrix Multiplication\n");
printf("   2 Simulate DTMC (pass <SAMPLES> parameter!)\n");
printf("   3 Convert CTMC to Uniformised DTMC\n");
printf("   4 Simulate CTMC (pass <TIME> parameter!)\n");
```

For `OP=2`, the user must also inform the number of samples at some point and for `OP=4`, the simulation time (in time units).

[1] Please consider doing this and then a subsequent Pull Request on the GitHub repository.

Suggestion: before looking at the solution, take all challenges and try to come up with your own solution.

4.1.2 Visual MC

Find a way of representing MC visually, using an auxiliary library or API like `graphviz` or some other.

4.1.3 Parallelisation of simulation samples

Vendors ship modern CPUs with more than one core. The idea of this project is to explore parallelism in simulations by distributing the computation of samples among multiple processing core and then assembling results altogether, showing to the modeller. This approach would make a lot of sense in huge models.

4.2 Models

4.2.1 Birth and Death model

Figure 4.1 shows a four states Birth and Death [15] process in CTMC with symbolic rates λ (for arrivals, i.e., births) and μ (for departures, or deaths).

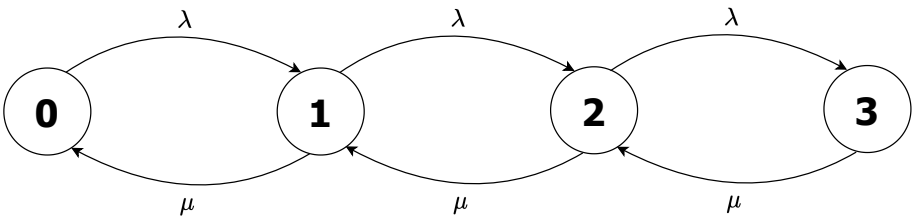


Figure 4.1: A four state Birth and Death CTMC model.

The Q matrix for this model follows a distinctive pattern:

$$\begin{array}{c} \\ \mathbf{0} \\ \mathbf{1} \\ \mathbf{2} \\ \mathbf{3} \end{array} \begin{array}{c} \begin{array}{cccc} \mathbf{0} & \mathbf{1} & \mathbf{2} & \mathbf{3} \end{array} \\ \begin{pmatrix} -\lambda & \lambda & 0 & 0 \\ \mu & -(\mu+\lambda) & \lambda & 0 \\ 0 & \mu & -(\mu+\lambda) & \lambda \\ 0 & 0 & \mu & -\mu \end{pmatrix} \end{array}$$

There are known equations that work with this type of models (closed form) [15, 14]. And also check out the model `S6-birth-and-death-ctmc.txt` in the GitHub repository for a six states Birth and Death process.

4.2.2 Mouse Maze model

This model was extracted from the book "Introduction to Probability" (page 440) [16]. The model employed DTMC and it has nine states, mapping the situation explained in Figure 4.2.

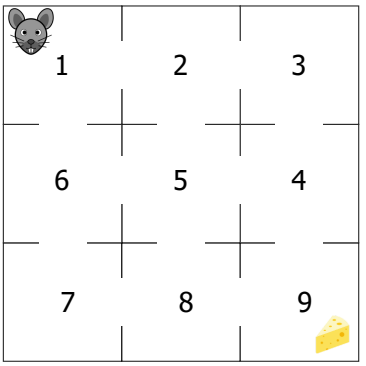


Figure 4.2: A mouse in a maze visiting cells mindlessly.

The model, as follows, shows the probabilities for reaching adjacent cells in the maze.

$$
\begin{array}{c} \\ 1 \\ 2 \\ 3 \\ 4 \\ 5 \\ 6 \\ 7 \\ 8 \\ 9 \end{array}
\begin{array}{c}
\begin{array}{ccccccccc} 1 & 2 & 3 & 4 & 5 & 6 & 7 & 8 & 9 \end{array} \\
\begin{pmatrix}
0 & \frac{1}{2} & 0 & 0 & 0 & \frac{1}{2} & 0 & 0 & 0 \\
\frac{1}{3} & 0 & \frac{1}{3} & 0 & \frac{1}{3} & 0 & 0 & 0 & 0 \\
0 & \frac{1}{2} & 0 & \frac{1}{2} & 0 & 0 & 0 & 0 & 0 \\
0 & 0 & \frac{1}{3} & 0 & \frac{1}{3} & 0 & 0 & 0 & \frac{1}{3} \\
0 & \frac{1}{4} & 0 & \frac{1}{4} & 0 & \frac{1}{4} & 0 & \frac{1}{4} & 0 \\
\frac{1}{3} & 0 & 0 & 0 & \frac{1}{3} & 0 & \frac{1}{3} & 0 & 0 \\
0 & 0 & 0 & 0 & 0 & \frac{1}{2} & 0 & \frac{1}{2} & 0 \\
0 & 0 & 0 & 0 & \frac{1}{3} & 0 & \frac{1}{3} & 0 & \frac{1}{3} \\
0 & 0 & 0 & \frac{1}{2} & 0 & 0 & 0 & \frac{1}{2} & 0
\end{pmatrix}
\end{array}
$$

Fractions are used so that every row sums to exactly 1; writing 0.33 would make rows sum to 0.99 and fail the line-sum test of Challenge 03.

The transition probabilities depend on the number of doors in the cell. For example, cells 1, 3, 7, and 9 have two doors, so the probability is $\frac{1}{2}$ whereas cells 2, 4, 6, and 8 have three doors each (probability of $\frac{1}{3}$) and finally, cell 5 has 4 doors (probability of $\frac{1}{4}$).

This model does have an analytical solution: for a random walk like this one, each cell's long-run probability is proportional to its number of doors. There are 24 doors in total, so

$$\pi = \tfrac{1}{24}\,[2,\ 3,\ 2,\ 3,\ 4,\ 3,\ 2,\ 3,\ 2],$$

i.e., the centre cell is visited most ($\approx 16.7\%$) and each corner least ($\approx 8.3\%$). The catch is that the chain is *periodic* (Section 1.9): cells $\{1, 3, 5, 7, 9\}$ only lead to cells $\{2, 4, 6, 8\}$ and vice versa, so the mouse alternates between the two groups at every

step. As a result, the Power matrix and VMM methods oscillate and never converge, even though π exists. Simulation still works, because it counts the fraction of visits over a long run. Alternatively, solve the "lazy" chain $\frac{1}{2}(P + I)$, in which the mouse stays put with probability $\frac{1}{2}$ at each step: it is aperiodic, has the same π, and VMM converges.

The mouse in the maze model is in GitHub's repository:

- File: `S9-maze-model-dtmc.txt`.

4.2.3 Availability model

Next model was discussed by Trivedi and Bobbio [3] (Example 9.2, page 311), about a blade server in a data centre. They investigated the availability (and conversely, the unavailability) using CTMC. The model is depicted in Figure 4.3.

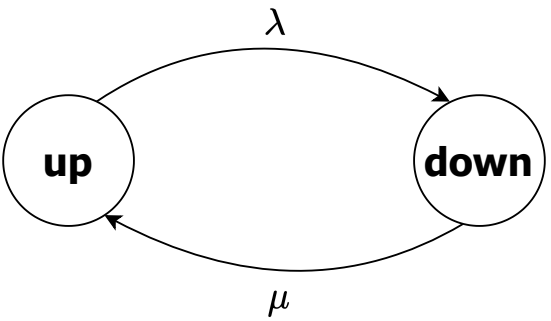


Figure 4.3: Availability modelling of a blade server in a data centre.

For this case, the IG of the CTMC is:

$$\begin{array}{c} \\ \textbf{up} \\ \textbf{down} \end{array} \begin{array}{c} \begin{array}{cc} \textbf{up} & \textbf{down} \end{array} \\ \begin{pmatrix} -\lambda & \lambda \\ \mu & -\mu \end{pmatrix} \end{array}$$

The authors solve this CTMC for its *transient* behaviour, i.e., how the probabilities change over time, using the Laplace transform method (they explain why in the book). If the server starts in the up state, the instantaneous availability, i.e., the probability that the server is up at time t, is

$$A(t) = \pi_{up}(t) = \frac{\mu}{\lambda + \mu} + \frac{\lambda}{\lambda + \mu}\, e^{-(\lambda+\mu)t}, \tag{4.1}$$

and the instantaneous unavailability is $U(t) = 1 - A(t) = \pi_{down}(t)$.

As $t \to \infty$ the exponential term vanishes, leaving the *steady-state* availability, which is what the methods in this book compute (e.g., by solving $\pi Q = 0$):

$$A = \pi_{up} = \frac{\mu}{\lambda + \mu} = \frac{MTTF}{MTTF + MTTR}, \qquad U = \pi_{down} = \frac{\lambda}{\lambda + \mu}, \tag{4.2}$$

where $MTTF = 1/\lambda$ is the mean time to failure and $MTTR = 1/\mu$ the mean time to repair.

4.2.4 Aging and Rejuvenation model

Next model was discussed by Huang et al. (1995) [1], about software aging and rejuvenation [17]. The basic idea behind these concepts is to employ preemptive mechanisms to restart applications to healthy states to prevent failures in the future. The authors have come up with a CTMC model depicted in Figure 4.4.

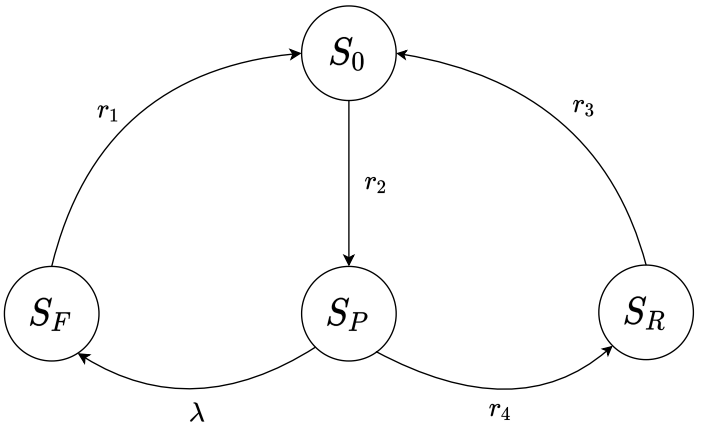


Figure 4.4: Software aging and rejuvenation model [1].

In the model, S_0 is the robust state of an application where it stays for a period equivalent to its base longevity interval, then it transitions to a failure probable state S_P. From this state (S_P), it goes to a failure state S_F where it remains until it is repaired. If only these states (S_0, S_P, and S_F) were considered, the model would correspond to a normal process without any rejuvenation.

The model as described in [1] uses the following assumptions (all rates per hour):

- $\lambda = \frac{1}{12\times30\times24} = 0.000115741$: failure rate from the failure-probable state S_P to failure S_F, i.e., a mean time to failure of 360 days (about 12 months) once the application has aged.
- $r_1 = 2$: recovery rate after an unexpected failure, i.e., a mean of 30 minutes to recover.
- $r_2 = \frac{1}{7\times24} = 0.005952381$: rate from the robust state S_0 to S_P, i.e., a base longevity interval of 7 days.
- $r_3 = 3$: repair rate after rejuvenation, i.e., a mean of 20 minutes.
- $r_4 = \frac{1}{(14-7)\times24} = 0.005952381$: rejuvenation rate from S_P to S_R; rejuvenating every 14 days, minus the 7 days already spent in S_0, gives a mean of 7 days in S_P.

$$
\begin{array}{c}
 \\
S_0 \\
S_F \\
S_P \\
S_R
\end{array}
\begin{array}{c}
\begin{array}{cccc}
S_0 & S_F & S_P & S_R
\end{array} \\
\begin{pmatrix}
0 & 0 & 0.005952381 & 0 \\
2 & 0 & 0 & 0 \\
0 & 0.000115741 & 0 & 0.005952381 \\
3 & 0 & 0 & 0
\end{pmatrix}
\end{array}
$$

The insertion of an intermediary state S_R that models a preemptive action from the probable failure state to replenish the application is inserted, as shown in the model. After this process is concluded, the application returns to the stable (robust) state. The rates for each state are discussed in the paper, the point the authors make is that rejuvenating applications costs less and helps applications stay (or return) in stable states more frequently. The seminal paper discusses how they modelled each rate and the reasoning behind it and other interesting notions about aging and rejuvenation. This idea has further implications in cyber-security, as studied by Czekster et al. (2021) [18].

It is worth pointing out that the model has only four states, modelling a CTMC that helps analysts consider different possibilities when designing or handling systems. Solving the model and having numerical outputs enormously help analysts understand behaviours and effectively cope with trade-offs.

Chapter 5

Final considerations

Markov Chains have multiple applications, and one hopes you have had a good journey reading this book, and your knowledge about the formalism has dramatically improved. Now it is possible to address more sophisticated problems using MC as another tool in your toolbox. However, don't think that because you now have a good *fancy* hammer (i.e., MC), every problem is a nail, and you must torture it until it fits the framework and reasoning discussed here. You should be able to apply MC depending on the problem at hand, the data you have, and the circumstances (the context) surrounding your analysis.

As one notices, the code present in the book (and in the repository) is far from perfect. If one looks deeper one may find a lot of defects and problems. It handles models with few number of states, having line sizes no greater than 1k (it does not even say that this is problem while trying to open). So, you could improve the code, translate it to C++ perhaps, add more controls, work with increased number of states, and so on. The sky is the limit – go for it!

The problems and models tackled here have reduced number of states, that is because it is targeted at educational purposes only. It is worth remarking that useful[1] real world modelling tends to require much larger order matrix to represent somewhat complex behaviours and relationships. Given all this, think that your *Markovian Journey* has just began.

5.1 What's next?

The next step will require further understanding of MC. For instance, one might:

- Investigate structured MC: Queueing Networks (QN) [19, 12], Stochastic Petri Nets (SPN), PEPA [20], or PTA/MDP.
- Understand and implement "Perfect Simulation" (also called *backwards* simulation) notions.

[1] Remember that quote from George Box, *"All models are wrong, some are useful."*

- Explore other related formalisms, such as Generalized Stochastic Petri Nets [21], Well Formed Networks, Layered Queueing Networks [22], Superposed Generalized SPNs (SGSPN) [23], and so on.

To deepen the knowledge in MC, it is suggested to give more time to PRISM and the inherent modelling formalisms thereof. It is a well-established and stable tool that researchers around the world use in a daily basis. You could for example explore other tool features such as investigate (in depth) other formalisms (suitable for CTMC or DTMC, PTA, and MDP), generation of automatic charts with variables, simulation, transient analysis, synchronising events. The tool comes with several examples and has a decent User Guide with lots of useful information to start modelling.

Using PRISM over the years one can attest its robustness and usefulness, however it is worth looking also into other tools. Mind that there is a wealth of tools to consider, for instance, the Modest Toolset (`https://www.modestchecker.net/`) or the Möbius Tool (`https://www.mobius.illinois.edu/`). And also, you may explore other quantitative analysis tools such as SHARPE (Symbolic Hierarchical Automated Reliability and Performance Evaluator) [24] – `https://sharpe.pratt.duke.edu/`.

You could start learning more about *dependability* reading the landmark paper by Avizienis et al. (2004) [25].Then, look at Jain's book on performance [14], the "Queueing networks and Markov chains" book [15], and Trivedi's "Reliability and Availability engineering" [3].

Your 'real' journey has just started!

About the author



Ricardo Melo Czekster is a curious individual based in Birmingham, United Kingdom (at least in 2022, time of writing this book). He promises he has no relation to Andrey A. Markov as his ancestors were from Vitebsk (Viciebsk) in Belarus. From what he has learned over time, Markov was from Ryazan, about 380 miles from Vitebsk.

In 1906, two major events happened in the world history: first, Markov published his *magnum opus* and second, avoiding impending wars, Ricardo's ancestors eventually settled down in Brazil in the southernmost region of Rio Grande do Sul, precisely in Santo Ângelo (distant 272 miles from the state capital, Porto Alegre).

He is a travelled and seasoned world visitor, having spent substantial time in Porto Alegre/Brazil (his hometown), Grenoble/France, Edinburgh/Scotland, Princeton/USA, St Andrews/Scotland, Newcastle upon Tyne/England and now Birmingham/England.

Ricardo has been studying Markov Chains since 2004 (there goes a lot of time!), whilst pursuing his PhD. Since then, he has done a lot of thinking and messing around this formalism, devising methods and solutions for tackling large state space models.

When he is not teaching or researching, he is doing some light manual woodworking, to try to forget about things for a while and engage in creating things out of wood and keep his fast-paced thought-stream in check.

Bibliography

[1] Y. Huang, C. Kintala, N. Kolettis, and N. D. Fulton, "Software rejuvenation: Analysis, module and applications," in *Symposium on Fault-Tolerant Computing*. IEEE, 1995, pp. 381–390.

[2] O. Häggström, *Finite Markov chains and algorithmic applications*. Cambridge University Press, 2002, vol. 52.

[3] K. S. Trivedi and A. Bobbio, *Reliability and availability engineering: modeling, analysis, and applications*. Cambridge University Press, 2017.

[4] A. L. Scherr, *An analysis of time-shared computer systems*. Massachusetts Institute of Technology, 1965, vol. 535.

[5] L. Page, S. Brin, R. Motwani, and T. Winograd, "The PageRank citation ranking: Bringing order to the web." Stanford InfoLab, Tech. Rep., 1999.

[6] A. N. Langville and C. D. Meyer, *Google's PageRank and beyond*. Princeton University Press, 2011.

[7] P. Von Hilgers and A. N. Langville, "The five greatest applications of Markov chains," in *Proceedings of the Markov Anniversary meeting*. Citeseer, 2006, pp. 155–158.

[8] K. L. Chung, "Markov chains," *Springer-Verlag, New York*, 1967.

[9] J. R. Norris, *Markov chains*. Cambridge university press, 1998, no. 2.

[10] S. P. Meyn and R. L. Tweedie, *Markov chains and stochastic stability*. Springer Science & Business Media, 2012.

[11] W. J. Stewart, *Introduction to the numerical solution of Markov chains*. Princeton University Press, 1994.

[12] ——, *Probability, Markov chains, queues, and simulation*. Princeton University Press, 2009.

[13] M. Kwiatkowska, G. Norman, and D. Parker, "PRISM 4.0: Verification of probabilistic real-time systems," in *International conference on computer aided verification*. Springer, 2011, pp. 585–591.

[14] R. Jain, *The art of computer systems performance analysis.* John Wiley & Sons, 1991.

[15] G. Bolch, S. Greiner, H. De Meer, and K. S. Trivedi, *Queueing networks and Markov chains: modeling and performance evaluation with computer science applications.* John Wiley & Sons, 2006.

[16] C. M. Grinstead and J. L. Snell, *Introduction to probability.* American Mathematical Soc., 1997.

[17] D. Cotroneo, R. Natella, R. Pietrantuono, and S. Russo, "A survey of software aging and rejuvenation studies," *ACM Journal on Emerging Technologies in Computing Systems (JETC)*, vol. 10, no. 1, pp. 1–34, 2014.

[18] R. M. Czekster, A. Avritzer, and D. S. Menasché, "Aging and Rejuvenation Models of Load Changing Attacks in Micro-Grids," in *2021 IEEE International Symposium on Software Reliability Engineering Workshops (ISSREW).* IEEE, 2021, pp. 17–24.

[19] F. Baskett, K. M. Chandy, R. R. Muntz, and F. G. Palacios, "Open, closed, and mixed networks of queues with different classes of customers," *Journal of the ACM (JACM)*, vol. 22, no. 2, pp. 248–260, 1975.

[20] J. Hillston, *A Compositional Approach to Performance Modelling*, ser. Distinguished Dissertations in Computer Science. Cambridge University Press, 1996.

[21] M. A. Marsan, G. Balbo, G. Conte, S. Donatelli, and G. Franceschinis, "Modelling with generalized stochastic Petri nets," *ACM SIGMETRICS Performance Evaluation Review*, vol. 26, no. 2, p. 2, 1998.

[22] G. Franks, T. Al-Omari, M. Woodside, O. Das, and S. Derisavi, "Enhanced modeling and solution of layered queueing networks," *IEEE Transactions on Software Engineering*, vol. 35, no. 2, pp. 148–161, 2009.

[23] S. Donatelli, "Superposed generalized stochastic Petri nets: definition and efficient solution," in *International Conference on Application and Theory of Petri Nets.* Springer, 1994, pp. 258–277.

[24] R. A. Sahner, K. Trivedi, and A. Puliafito, *Performance and reliability analysis of computer systems: an example-based approach using the SHARPE software package.* Springer Science & Business Media, 2012.

[25] A. Avizienis, J.-C. Laprie, B. Randell, and C. Landwehr, "Basic concepts and taxonomy of dependable and secure computing," *IEEE Transactions on Dependable and Secure Computing*, vol. 1, no. 1, pp. 11–33, 2004.

Index

A
aggregation, 13
aperiodic, 11

B
Belfast model, 17

C
CTMC, 5, 29
Czekster, 51

D
digraph, 4
DTMC, 5
DTMC simulation, 9
duration, 7

E
eigenvector, 4
embedded DTMC, 32
ergodic, 12, 31
ergodicity, 4
exponential, 7

G
Google PageRank algorithm, 4, 22

I
Infinitesimal Generator, 29
irreducible, 11

L
level-of-detail, 4
Lily Pad model, 33
Linear Congruential Generator, 9
lumpability, 13

M
Markov Chains, 5
Markov Chains, MC, 3
Markov Process, 5
Markov property, 7
model, 3

N
Nearly Completely Decomposable, 13

P
Power Matrix, 15
preconditioning, 16
PRISM model, 23
pseudo-random, 9

R
residence time, 5

S
seed parameter, 13
sojourn time, 5
stochastic Petri nets, 13, 49
structured MC, 49
system, 3

T
time shared system, 4
transitions, 4

U
uniformised DTMC, 31

V
Vector-Matrix Multiplication, 16



Yet another venture by *the lazy panda collection*
URL: `https://pixabay.com/photos/mammal-wildlife-animal-zoo-panda-3074618/`
Simplified Pixabay License

Markov Chains for programmers
Copyright 2022- Ricardo M. Czekster

Why should you read this book?

- It is directed at programmers (at any level) interested in Markovian numerical methods.
- It addresses simple Markov Chains and explains the numerical methods behind the formalism.
- It explains simple models and discusses trade-offs about computation effort concerning solutions.